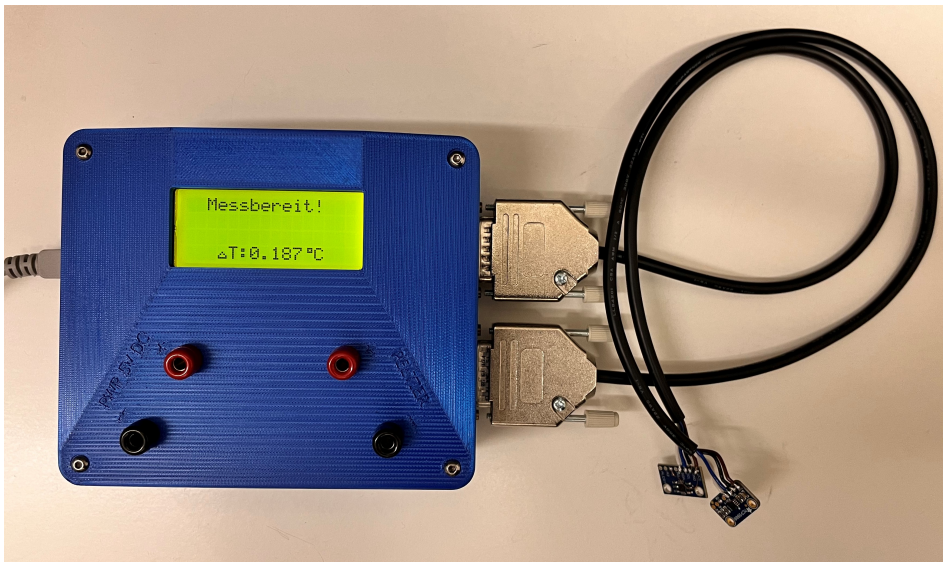
Peltier-Messgerät



Datum: Juli 2022
Authoren: Michael Oblak, 01619042
Philipp Rohr, 11706159

Inhaltsverzeichnis

1 Funktionsbeschreibung des Projekts **3**

2 Beschreibung der verwendeten Hardware **4**
2.1 Temperatursensoren 5
2.2 ADC 6
2.3 Display 6
2.4 3D-Druck 7
2.5 Messaufbau 10

3 Beschreibung der verwendeten Software **11**
3.1 Arduino IDE 11
3.2 Python 12

4 Auswertung und Darstellung **13**
4.1 RC12-8L 14
4.2 RC6-6L 15
4.3 RC3-8L 16

5 Diskussion der Ergebnisse **17**

6 Anhang mit gut dokumentierten Sourcecode **18**
6.1 Arduino Code 18
6.2 Python Code 30

7 Anhang **34**
7.1 Datenblatt Base Shield 34
7.2 Datenblatt LCD-Module 34
7.3 Datenblatt NMosfet 34
7.4 Datenblatt Sub-D 34
7.5 Zeichnungen für 3D-Druck 34

1 Funktionsbeschreibung des Projekts

Es wurde ein Messgerät realisiert, welches den Widerstand eines Peltier-Elements über die Temperatur innerhalb einer zuvor definierten Zeit misst. Die Messung wird per UART in Python aktiviert, dabei werden die Temperaturen an der Ober- und Unterseite des Peltier-Elements gemessen. Währenddessen wird an 3 Punkten die Spannung gemessen, wodurch der Spannungsabfall am Peltier-Element $U_{Peltier}$ und der Spannungsabfall am Shunt-Widerstand U_{Shunt} mit $R = 2.025\ \Omega$ bestimmt wird. Dabei wird der Strom I durch das System mittels folgender Gleichung bestimmt.

$$I = \frac{U_{Shunt}}{R} \tag{1}$$

Mit diesem errechneten Strom wird der temperaturveränderliche Widerstand des Peltier-Elements mittels folgender Gleiching bestimmt.

$$R_{Peltier}(T) = \frac{U_{Peltier}(T)}{I} \tag{2}$$

Das Programm kann per UART in Python gestartet werden, wobei zuvor eine Messzeit deklariert werden muss. Dabei werden der Messzeitpunkt in Sekunden, die Peltier-Leerlaufspannung in Volt, die Peltierspannung in Volt, der Strom in Ampere, der temperaturabhängige Widerstand des Peltier-Elements, sowie die beiden Temperaturen (Oberseite und Unterseite vom Peltier-Element) an Python übergeben. Die Auswertung erfolgt unmittelbar nach Abschluss der Messung, und es erfolgt eine Ausgabe eines Plots inklusive der gefitteten Funktion am Computer.
Das Display zeigt während dem Messvorgang die live Daten von $U_{Peltier}$, I_{Shunt}, $R(T)$ und die Temperaturdifferenz ΔT der beiden Temperatursensoren. Um zu gewährleisten, dass es zu keinen Fehlmessungen kommt, wird zu Beginn und während dem Messen überprüft, ob eine aufrechte Verbindung mit den Temperatursensoren vorhanden ist. Im Falle eines Verbindungsverlustes wird die Messung abgebrochen und eine Fehlermessung am Display angezeigt.

2 Beschreibung der verwendeten Hardware

Tabelle 1: Hardewareliste

Hardware	**Name**	**Hersteller**	**Eigenschaften**
Mikrocontroller Board	ATmega328	AZDelivery	
Base Shield	Base Shield V2	Seeed Studio Grove	
ADC	ADS1015	Adafruit	12-Bit
Temperatursensor	MCP9808	Adafruit	
Kühlkörper			
Widerstände			$R = 2.025\ \Omega$
Widerstände			$R = 10\ \text{k}\Omega$
Widerstände			$R = 390\ \Omega$
Widerstände			$R = 220\ \Omega$
Mosfet	IRF540ZS	isc	NMOS
Netzgerät	EA-PS 3032-05 B	EA Elektro-Automatik	
Display	LC 1604		16 x 4
Bananenstecker	MLN 50/2,5 Au	Hirschmann	4mm
Bananensteckerbuchsen			
Druckfedern			
Gehäuse			3D-Druck
SUB-D Stecker und Buchse	RS PRO Sub-D Steckverbinder	RS	15 polig, 2 zeilig
Peltier-Element	RC12-8L	Marlow Industries	
Peltier-Element	RC6-6L	Marlow Industries	
Peltier-Element	RC3-8L	Marlow Industries	
Peltier-Element	RC3-6-01L	Marlow Industries	

Das Mikrocontroller Board ist wie in Abbildung 1 mit denen in Tabelle 1 ersichtlichen elektrischen Bauteilen verbunden. In den folgenden Unterkapiteln werden die Verbindungen der Temperatursensoren, dem ADC und des Displays detailliert beschrieben.

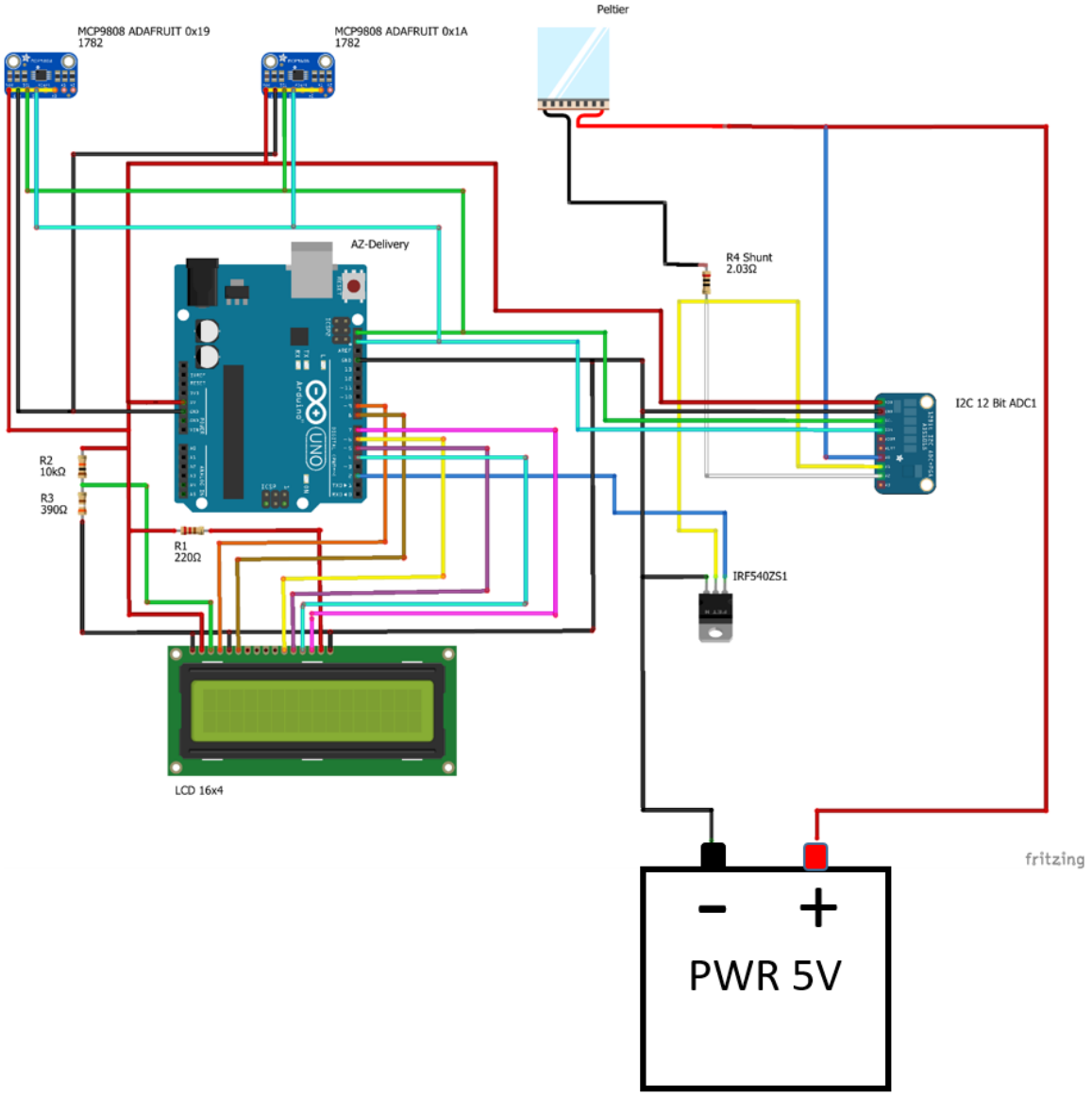


Abbildung 1: Verdrahtungsskizze der verwendeten Komponenten

2.1 Temperatursensoren

Die Temperatursensoren sind via I2C am Base Shield mit dem Mikrocontroller Board verbunden. Hier wurde VCC des Mikrocontroller Board mit VDD des Sensors verbunden, weiters wurde GND zu GND, SCL zu SCL und SDA zu SDA verbunden und die Kabel direkt an den Sensor gelötet.
Um die Temperatursensoren jederzeit vom Messgerät abstecken zu können, wurden SUB-D Buchsen am Rande des Gehäuses befestigt, sowie die dazu passenden SUB-D Stecker an den

Temperatursensoren gelötet. Die SUB-D Stecker sind 15-polig und 2 zeilig, von den 15 Pins wurden die Pins 1, 3, 5 und 14 benutzt (genaue Positionen siehe Abbildung 1). Um den Temperatursensoren die jeweilige Adresse zuordnen zu können, wurde bei einem der beiden Adafruit MCP9808, VDD mit A0 verbunden (0x19 Adresse) und beim anderen Temperatursensor VDD mit A1 verbunden (0x1A).[1]



Abbildung 2: SUB-D Stecker mit den verwendeten Pins 1, 3, 5 und 14 beginnend rechts oben mit 1

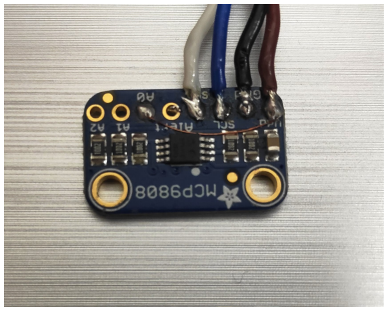


Abbildung 3: Adafruit MCP9808, VDD ist mit A0 verbunden

2.2 ADC

Der ADC ist wie die Temperatursensoren via I2C auf dem Base Shield, mit dem Mikrocontroller Board verbunden und dabei wurden wieder VCC zu VDD, GND zu GND, SCL zu SCL und SDA zu SDA verbunden. Der ADC besitzt die Adresse 0x18, da keiner der Ausgänge mit VDD (Standardkonfiguration) verbunden ist. Die 3 Ausgänge A0, A1, und A2 wurden benutzt um die jeweiligen Spannungen zu messen. Hier misst A0 die Spannung vor und A2 die Spannung nach dem Peltier-Element. A1 misst dann die Spannung nach dem Shunt-Widerstand, zu sehen am Gesamtschaltplan Abbildung 1. [2]

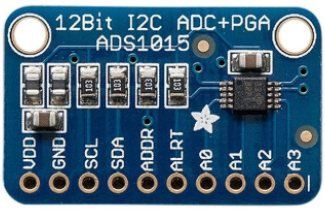


Abbildung 4: ADS1015

2.3 Display

Das Display wurde wie in Abbildung 5 an das Mikrocontroller Board angeschlossen. Es wurden 3 Widerstände mit $R_1 = 220\ \Omega$, $R_2 = 10k\ \Omega$ und $R_3 = 390\ \Omega$ verwendet um den Kontrast des Dislays ohne Potentiometer einzustellen. Anders als in der Grafik zu sehen, handelt es sich beim Display am Messgerät um ein 16x4 Display, solch ein Bauteil war aber nicht unter Fritzing [5] erhältlich, weshalb zur Darstellung des Displays im Schaltplan ein 16x2 Display verwendet wurde. [3]

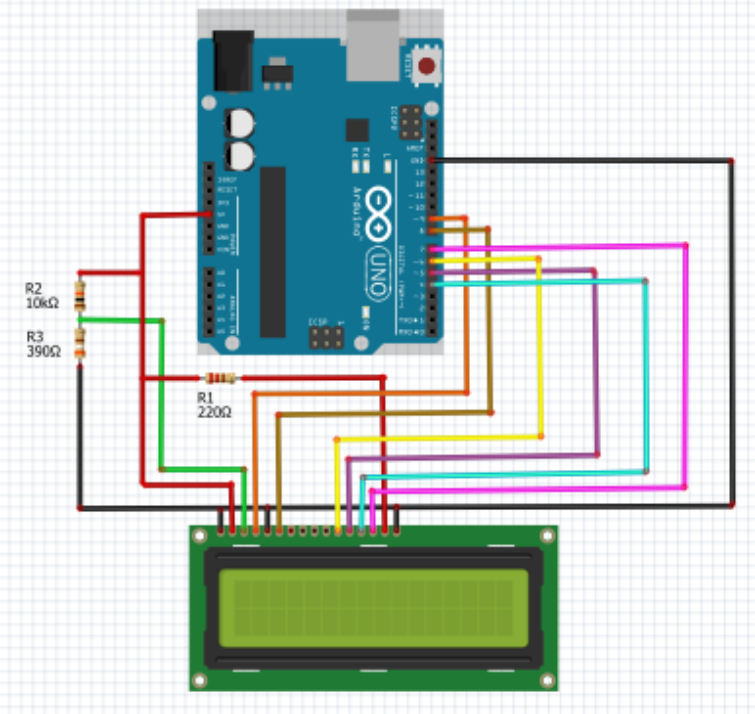


Abbildung 5: Display an Arduino

2.4 3D-Druck

Das Gehäuse wurde mittels 3D-Druck selbst konstruiert und von der Werkstätte des Physik-Institutes der Uni Graz gedruckt. Es besteht aus mehreren Komponenten: Gehäuseboden, Gehäusedeckelmittelteil, Gehäusedeckel, Befestigungsblock und Schalter.

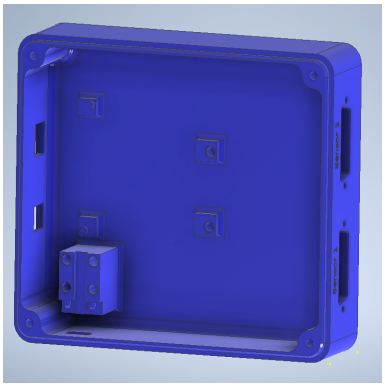


Abbildung 6: Gehäuseboden

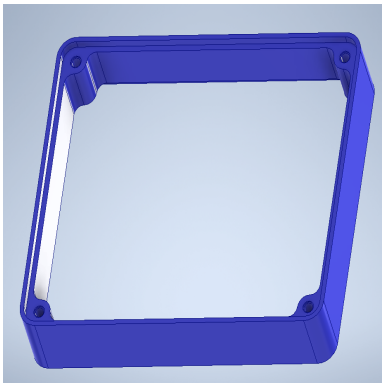


Abbildung 7: Mittelteil

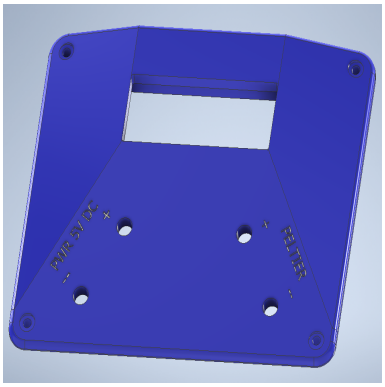


Abbildung 8: Gehäusedeckel

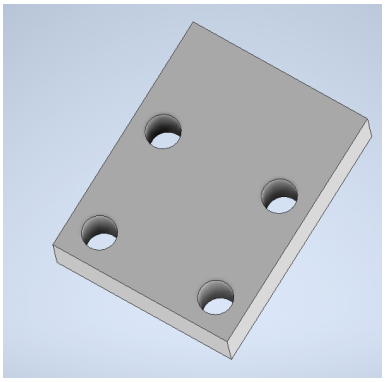


Abbildung 9: Befestigungsblock

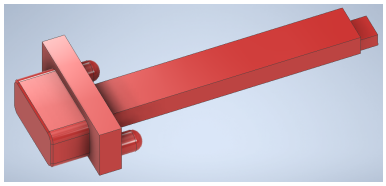


Abbildung 10: Schalter

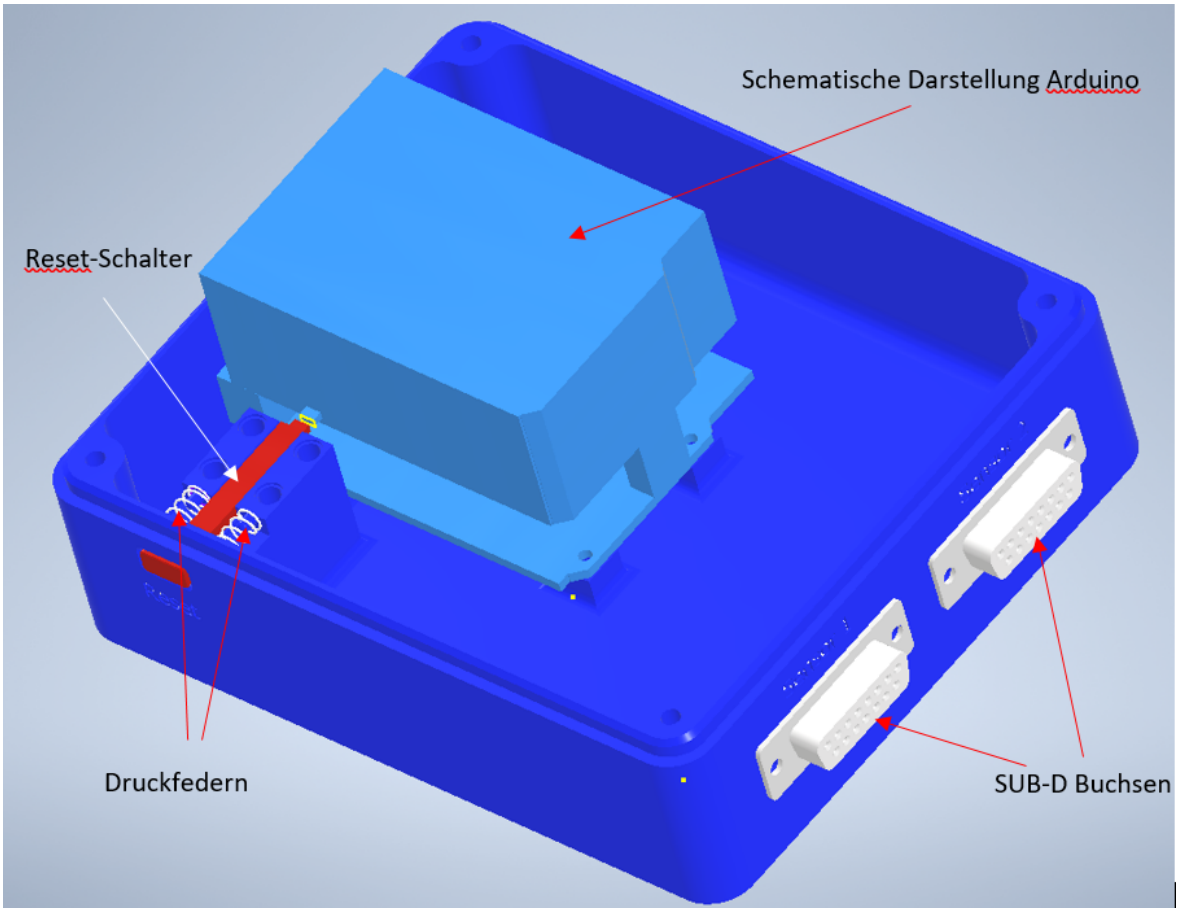


Abbildung 11: Baugruppe des Gehäusebodens mit Mikrokontroller Board, Reset-Schalter, SUB-D Buchsen und den Druckfedern ohne Verdrahtung

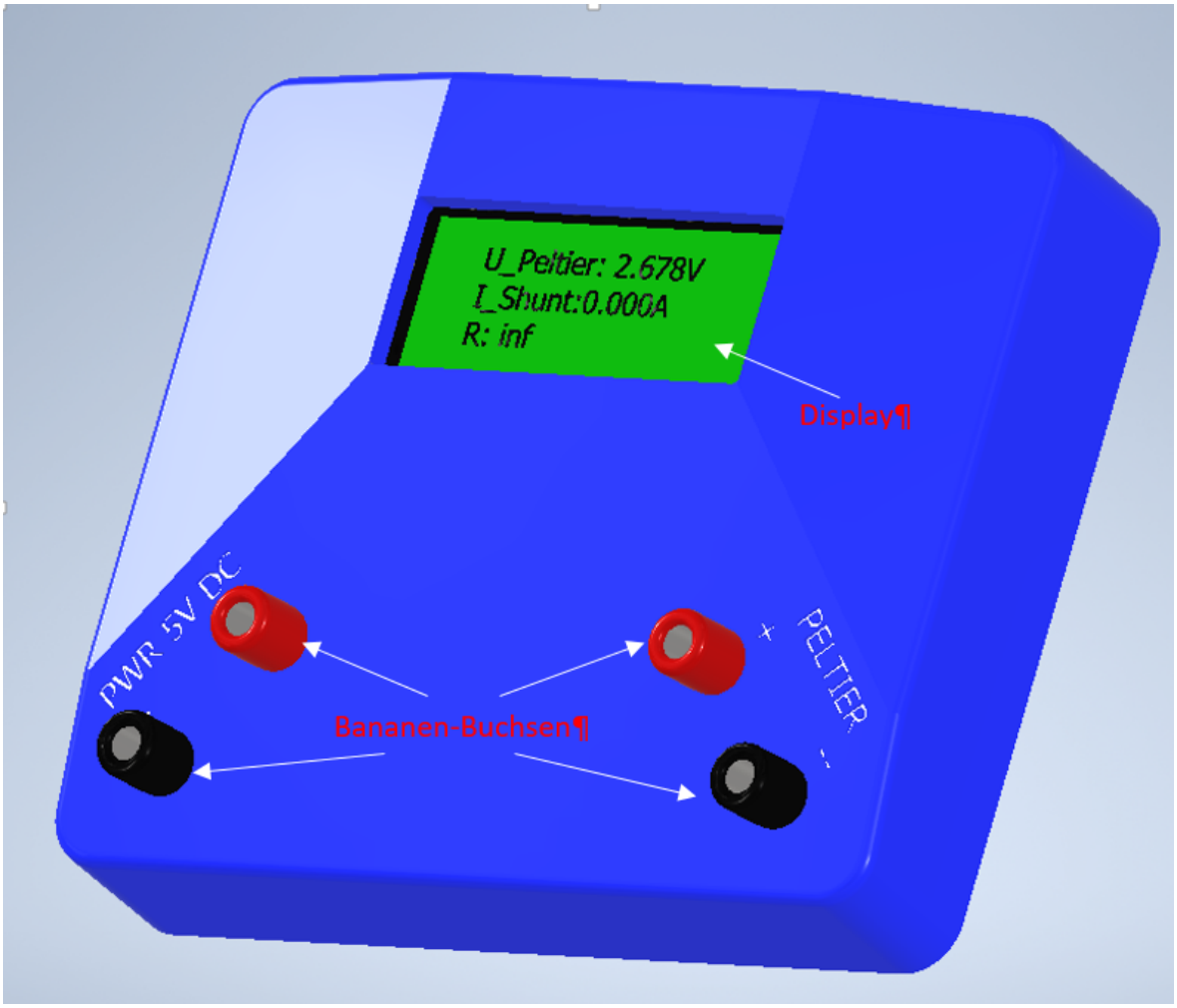


Abbildung 12: Baugruppe Deckel mit Display, und den schwarzen, sowie roten Bananenbuchsen

2.5 Messaufbau

Das Peltier-Element geht mittels der Wärmeleitpaste eine thermische Verbindung mit dem Kühlkörper ein. Auf den Kühlkörper wurden die selbst 3D-gedruckten Haltevorrichtungen angebracht, welche mittels Schrauben die Temperatursensoren am Peltier-Element beziehungsweise am Kühlkörper befestigen.

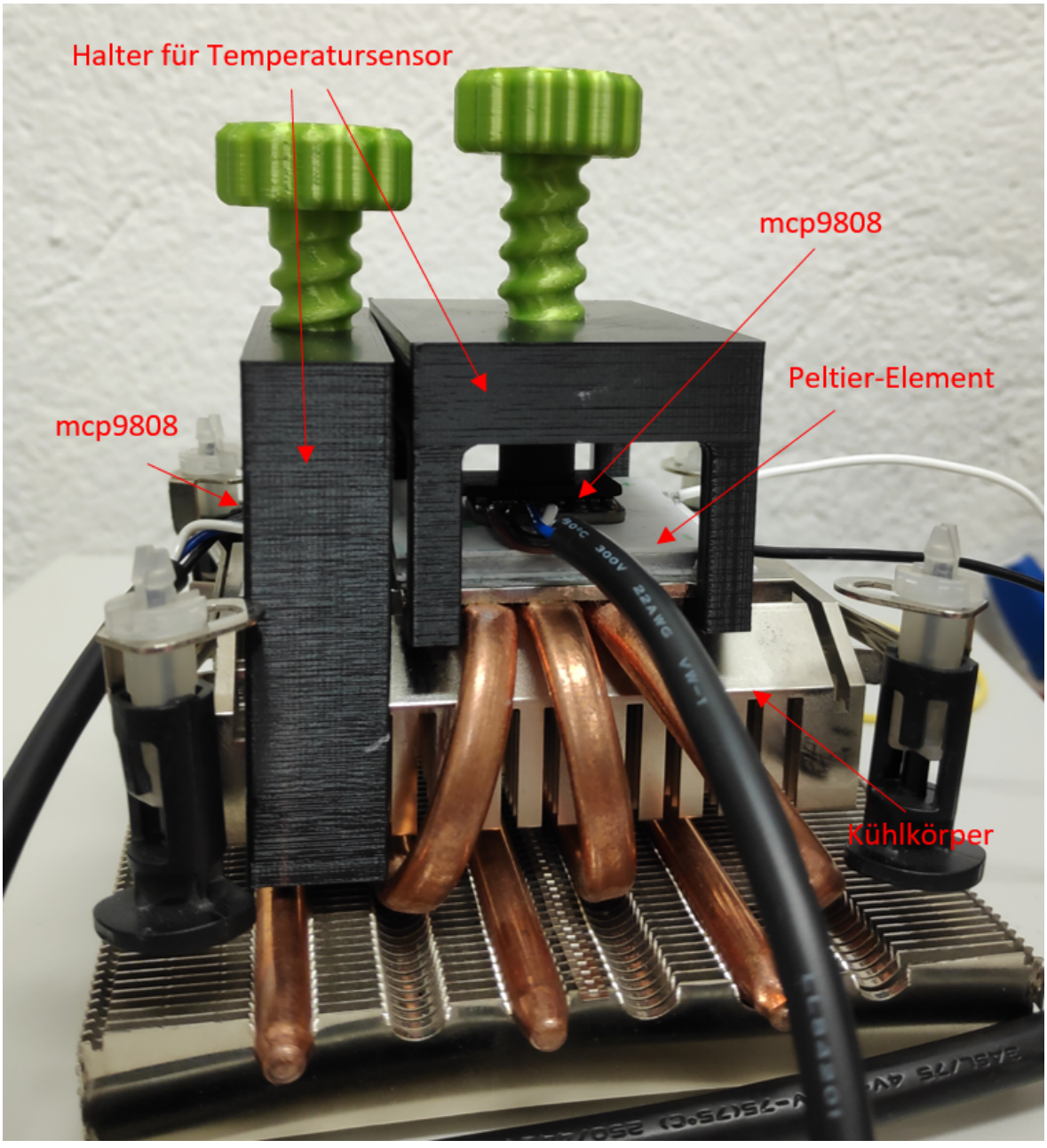


Abbildung 13: Messaufbau mit Haltevorrichtungen, Temperatursensoren, Peltier-Element und Kühlkörper

3 Beschreibung der verwendeten Software

3.1 Arduino IDE

Das Arduino Programm startet mit dem Hinzufügen der benötigten Libraries und dem erstellen der beiden Temperatursensoren, sowie dem Deklarieren von globalen Variablen.

In der Funktion *tempcheck*, welche im void Setup aufgerufen wird, wird überprüft, ob die Temperatursensoren angeschlossen sind. Wenn dies nicht der Fall ist, gibt das Display, sowie der Serial Monitor eine Fehlermeldung aus. Diese zeigt, welcher der beiden Temperatursensoren fehlt oder ob beide fehlen. Ebenfalls wird in dieser Funktion die Auflösung der Temperatursensoren festgelegt.

Void messen wird in der Funktion *ProcessReceivedData* aufgerufen. Sie ist die Funktion, welche sich um den Messvorgang und die Ausgabe am Display und Serial Monitor kümmert. Es wird die Startzeitverzögerung in Millisekunden initialisiert, und die zum Messen benötigten Variablen erstellt. Der N-Mosfet wird durchgeschalten und die Temperaturen und Spannungen werden gemessen. Der Spannungsabfall am Peltier, der Strom durch das System, der Temperaturunterschied und der Widerstand des Peltier-Elements werden direkt berechnet und am Display über die Funktion *DisplayMeasurementData* ausgegeben. Anschließend schließt der Mosfet und die Leerlaufspannung wird gemessen.
Am Serial Monitor werden die Messzeit, die Leerlaufspannung, der Spannungsabfall am Peltier-Element, der Strom durch das System, der Widerstand des Peltier-Elements und die beiden Temperaturen ausgegeben. Die Messung wird für eine vorgegebene Zeit lang durchgeführt.

Um das Programm zu starten, muss im Serial Monitor 'messen' und die Messzeit angegeben werden. Die Funktionen *ProcessUART* und *ProcessReceivedData* stammen von Jan Enekel, und überprüfen die im Serial Monitor eingetippten Befehle. *ProcessUART* holt sich den eingegebenen Befehl und zerlegt ihn in characters. *ProcessReceivedData* überprüft anschließend die characters mit den Funktionen *hasStringInside* und *getCharElementFromString*, ob 'messen' und eine Messzeit angegeben wurden. Falls nicht, wird eine Fehlermeldung ausgegeben, sonst wird die Funktion *messen* gestartet.

Zwischen den Messvorgängen läuft die Funktion *DisplayZwischenMessung*, sie gibt am Display die aktuelle Temperaturdifferenz an und gibt Bescheid, wenn die Messung wieder startbereit ist. Ebenfalls wird durchgehend überprüft, ob die Temperatursensoren angeschlossen sind.

Im **void setup** werden das Display und der ADC aktiviert und die Datenrate in Bit pro Sekunde (Baud) für die serielle Datenübertragung festgelegt. Des Weiteren wurden Sonderzeichen für das Display konstruiert.
Im **void loop** werden dann die Funktionen *ProcessUART*, *ProcessReceivedData* und *DisplayZwischenMessung* durchlaufen. [4]

3.2 Python

Nach dem Laden der Libraries, wird die Funktion *write_read* definiert, diese importiert die Daten vom Serial Monitor des Arduino IDE nach Python.

Nach Initialisieren des Mikrocontroller Board wird ein input Befehl definiert, der beim Programmstart die Laufzeit des Programms abfragt und vom Anwender eingegeben wird. Anschließend beginnt eine while-Schleife, welche die Funktion *write_read* aufruft und erst stoppt wenn vom Mikrocontroller Board die Nachricht 'Messung fertig!' eintrifft. Folgend werden Variablen initialisiert, in welche dann mittels for-Schleife die Messwerte gespeichert werden.

Zum Veranschaulichen der Messergebnisse wird ein Exponential-Fit und ein Polyfit erstellt und zusammen mit den Messwerten geplottet. Der ausgebende Plot gibt den temperaturabhängigen Widerstand von Peltier-Elementen mit Achsenbeschriftung zurück. Ebenfalls werden die Koeffizienten vom Polynomfit und Exponentialfit in der Konsole ausgegeben.

4 Auswertung und Darstellung

Die Messung wurde mit 3 verschiedenen Peltier-Elementen durchgeführt. Der Kühlkörper, sowie der restliche Versuchsaufbau blieben gleich. Die Messdauer betrug bei allen Peltier-Elementen 30 Sekunden.



Abbildung 14: Peltier-Elemente: RC12-8L, RC6-6L, RC3-8L von links nach rechts

4.1 RC12-8L

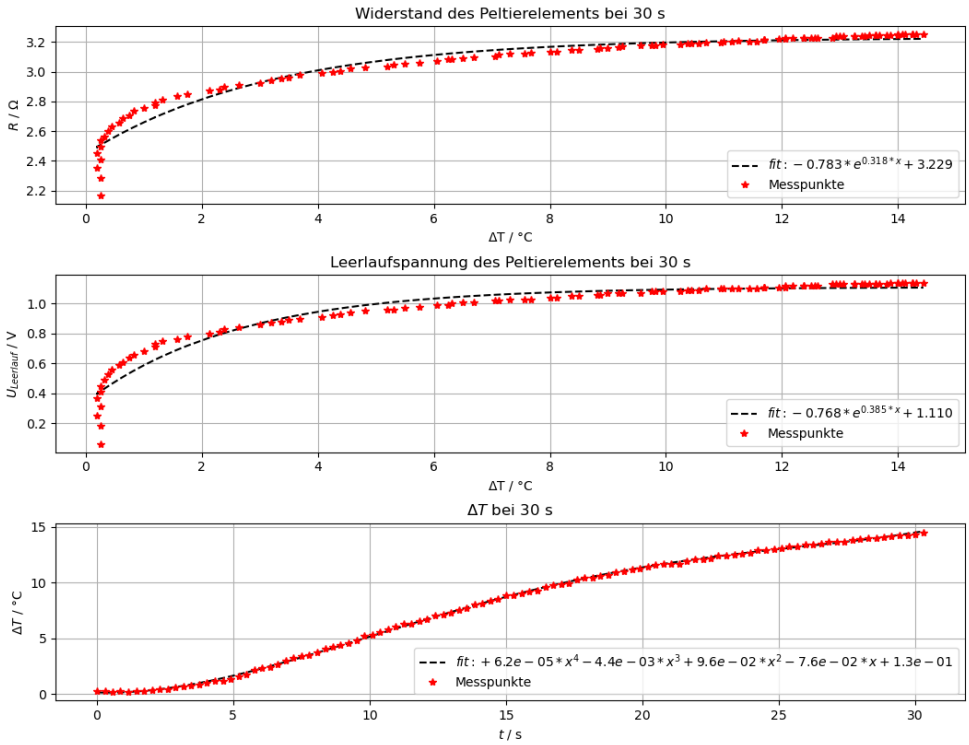


Abbildung 15: Peltier-Element RC12-8L

Das RC12-8L ist das flächenmäßig größte der 3 Peltier-Elemente. Wie in Abbildung 15 zu sehen ist, zeigt es den kleinsten Temperaturunterschied in der vorgegebenen Zeit. Dafür weißt es die größte Leerlaufspannung und den größten Widerstand bei gleicher Temperaturdifferenz auf.

4.2 RC6-6L

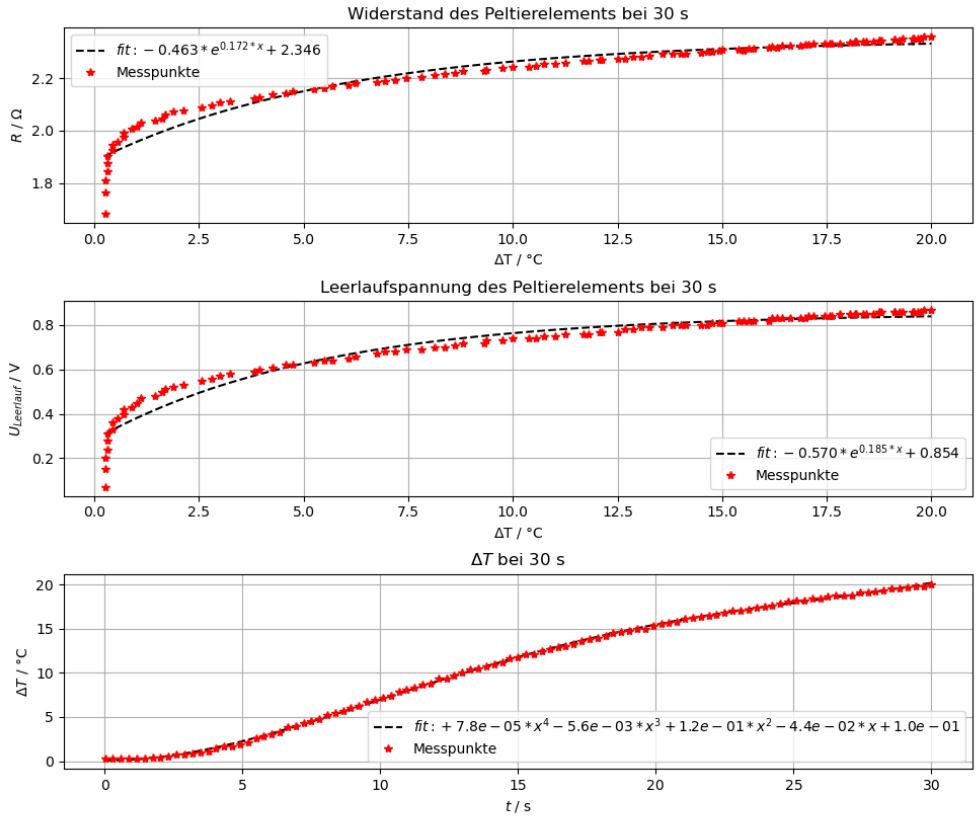


Abbildung 16: Peltier-Element RC6-6L

Das RC6-6L ist das flächenmäßig zweitgrößte der 3 Peltier-Elemente. Wie in Abbildung 17 zu sehen ist, weißt die Temperaturkurve des RC6-6L einene steilere Steigung als die des RC12-8L auf.Außerdem zeigt es eine geringere Leerlaufspannung und kleineren Widerstand bei gleicher Temperaturdifferenz.

4.3 RC3-8L

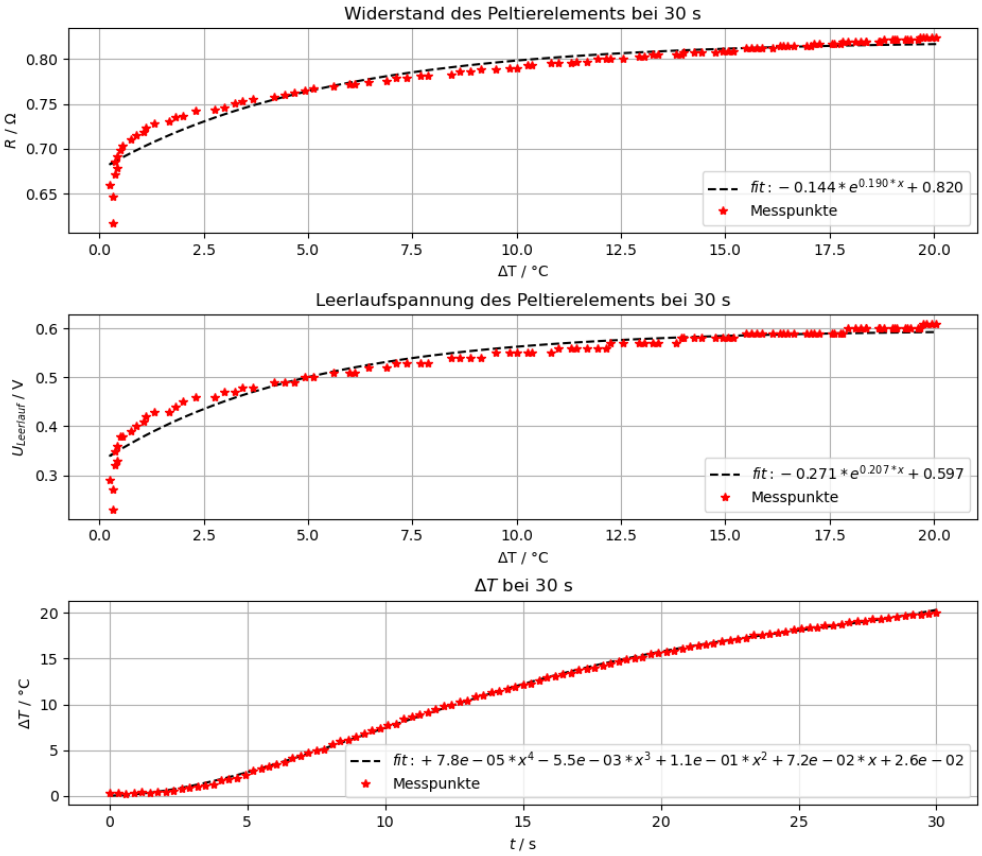


Abbildung 17: Peltier-Element RC3-8L

Das RC3-8L ist das flächenmäßig kleinste der 3 Peltier-Elemente. In Abbildung 16 sieht man, dass das RC4-8L einen steileren Temperaturanstieg als das RC12-8L besitzt, jedoch besitzt es einen ähnlich großen Temperaturanstieg wie das RC6-6L bei einer Messzeit von 30 Sekunden. Des Weiteren weißt es die geringste Leerlaufspannung und den kleinsten Widerstand bei gleicher Temperaturdifferenz im Vergleich zu den beiden flächenmäßig größeren Peltier-Elementen auf.

5 Diskussion der Ergebnisse

Eine mögliche Verbesserung im Code, wäre die Möglichkeit auf einen bestimmten Temperaturunterschied hin zu messen, anstatt auf eine vorherbestimmte Zeit. Dies müsste jedoch mit einer Temperaturobergrenze geschehen, falls ein zu hoher Temperaturunterschied angegeben wird.
Des Weiteren fehlt eine Unsicherheitsanalyse, welche aber im Nachhinein ergänzt werden kann, wenn das Messgerät zum Einsatz kommt.

Eine weitere Verbesserungsmöglichkeit wäre von Python aus die Messung abzubrechen, denn das Messgerät wiederholt derzeit die Messung mehrmals, wenn die Messung von Python aus gestartet wird. Dies verzögert eine erneute Messung, da der Messaufbau erst einen Temperaturausgleich vollführen muss, bevor es wieder zur Messung benutzt werden kann.
Der Polyfit stimmt erst ab einer sehr großen Ordnung mit den gemessenen Kurven für $R/\Delta T$ und $U_{Leerlauf}/\Delta T$ überein, weshalb hier nur der Exponentialfit verwendet wurde. Für $\Delta T/t$ wurde hingegen ein Polynomfit 4. Grades verwendet.

Große Probleme bereitete der Temperatursensor CJMCU-9808 von TECNOIOT, welcher während der Auswertung plötzlich nicht mehr funktionierte. Dieser wurde dann mit einem Temperatursensor von Adafruit, dem MCP9808, ersetzt.

6 Anhang mit gut dokumentierten Sourcecode

6.1 Arduino Code

```
//=============================================================================================
//Aufruf Libraries
//=============================================================================================
#include <Wire.h> //I2C
#include <Adafruit_ADS1X15.h> //ADC 12 bit
#include "Adafruit_MCP9808.h" //Temperatursensor
#include <LiquidCrystal.h> //Display

//=============================================================================================
Adafruit_MCP9808 tempsensor1 = Adafruit_MCP9808(); // Definition des Temperatursenor 1
Adafruit_MCP9808 tempsensor2 = Adafruit_MCP9808(); // Definition des Temperatursenor 2

LiquidCrystal lcd(9, 8, 6, 5, 4, 7); //verwendete Pin-Positionen

char str_T1[8]; //initialisieren zum Abspeichern als String für T1
char str_T2[8]; //initialisieren zum Abspeichern als String für T1
char str_delta_T[8]; //initialisieren zum Abspeichern als String für delta_T

// UART Kommunikation - Globale Variablen
bool newData = false;
const byte numChars = 96;
char string[numChars]; // Array um Daten vom Serial Interface abzuspeichern
char receivedChars[numChars]; // Array um erhaltene Daten vom Serial Interface speichern


Adafruit_ADS1015 ads; //Definition des ADS als ads

float R_shunt = 2.025; //Ohm, Shuntwiderstand
int PWM_PIN = 2; //Pin an Mosfet
//int pwmval = 0; //initialisieren der Variable

#define MY_IDENTITY "PeltierMessgerät 1v0"

//=============================================================================================
//Funktion tempcheck
//=============================================================================================
void tempcheck () {
    //Wenn Tempsensor 1 und Tempsensor 2 nicht angeschlossen sind,
    //kommt eine Fehlermeldung im Serialmonitor und am LCD
  if (!tempsensor1.begin(0x19) && !tempsensor2.begin(0x1A)) {
    Serial.println("Couldn't find Tempsensor 1 and Tempsensor 2!"
                   " Check your connections and restart.");
```

```
  //LCD-Ausgabe des Fehlers
  lcd.setCursor(0,0); //Curserposition auf 1.Zeile, 1.Feld
  lcd.print("Problem:"); //Ausgabe des Fehlers auf 1.Zeile, 1.Feld
  lcd.setCursor(0,1); // Curserposition auf 2.Zeile, 1.Feld
  lcd.print("Check: T1");
  lcd.setCursor(0,2);
  lcd.print("Check: T2");
  lcd.setCursor(0,3);
  lcd.print("and restart");

  while (1);

}

//Wenn Tempsensor 1 nicht angeschlossen ist Tempsensor 2 schon,
//kommt eine Fehlermeldung im Serialmonitor und am LCD
if (!tempsensor1.begin(0x19)) {
  Serial.println("Found Tempsensor 2 \nCould not find Tempsensor 1)! "
                 " Check your connections and restart.");

  //LCD-Ausgabe des Fehlers
  lcd.setCursor(0,0);
  lcd.print("Problem:");
  lcd.setCursor(0,1);
  lcd.print("Found: T2");
  lcd.setCursor(0,2);
  lcd.print("Check: T1");

  while (1);

}

//Wenn Tempsensor 2 nicht angeschlossen ist Tempsensor 1 schon,
//kommt eine Fehlermeldung im Serialmonitor und am LCD
if (!tempsensor2.begin(0x1A)) {
  Serial.println("Could not find Tempsensor 2! "
                 "Check your connections and  restart.");


  //LCD-Ausgabe des Fehlers
  lcd.setCursor(0,0);
  lcd.print("Problem:");
  lcd.setCursor(0,1);
  lcd.print("Found: T1");
  lcd.setCursor(0,2);
  lcd.print("Check: T2");
```

```
    while (1);
  }

  // Setzt die Auflösung der Temperatur wie in Tabelle darunter
  tempsensor1.setResolution(3);
  tempsensor2.setResolution(3);

  // Mode Resolution SampleTime
  //  0    0.5°C       30 ms
  //  1    0.25°C      65 ms
  //  2    0.125°C     130 ms
  //  3    0.0625°C    250 ms
}

//=============================================================================================
//Funktion messen
//=============================================================================================

void messen( float messzeit )
{
        //Initialisieren der Variablen
        bool done = false; //gebraucht für Beendigung der Messung nach Messzeit
        int16_t A0_digits = 0;
        int16_t A1_digits = 0;
        int16_t A2_digits = 0;

        float A0_Spannung = 0.0;
        float A1_Spannung = 0.0;
        float A2_Spannung = 0.0;

        float T1 = 0.0;
        float T2 = 0.0;

        float U_Peltier = 0.0;
        float I_shunt = 0.0;
        float R = 0.0;
        float U_PeltierLeerlauf = 0.0;

        //Initialisieren der Startzeitverzögerung in Millisekunden
        long starttime = millis();  //Starte erste Zeitmessung
        long actualtime = 0; //actualtime auf 0


        digitalWrite(PWM_PIN,HIGH);     // aktivere den NMOS

        int i = 0;
```

```
do //mach Messung bis done=true
{
    // Messung der Digits des ADC ADS1015 12BIT
    A0_digits = ads.readADC_SingleEnded(0);
    A1_digits = ads.readADC_SingleEnded(1);
    A2_digits = ads.readADC_SingleEnded(2);

    // Umrechnen der Digits aus Spannung
    A0_Spannung = ads.computeVolts(A0_digits);
    A1_Spannung = ads.computeVolts(A1_digits);
    A2_Spannung = ads.computeVolts(A2_digits);

    // Messung Temperatur
    T1 = tempsensor1.readTempC(); //Auslesen von Temperatursensor 1
    T2 = tempsensor2.readTempC(); //Auslesen von Temperatursensor 2

    // überprüfe ob T1 oder T2 nan wiedergibt -> ob Sensor angeschlossen
    if(isnan(T1) || isnan(T2))
    {
      Serial.println("Sensor verbindung getrennt!");
    }

    // Berechnung von U_Peltier, I_Shunt und R(T)
    U_Peltier = A0_Spannung - A2_Spannung;
    I_shunt = (A2_Spannung - A1_Spannung) / R_shunt;
    R = U_Peltier / I_shunt;

    float delta_T = abs(T1-T2); // Berechnung für Displayausgabe

    actualtime = millis() - starttime; //Berechne der Zeitdifferenz

    ////////////////////////////////
    // Leerlaufsspannungsmessung //
    ////////////////////////////////

    digitalWrite(PWM_PIN,LOW);     // deaktivieren den NMOS
    delay(3);
    // Messung der Digits
    A0_digits = ads.readADC_SingleEnded(0);
    A2_digits = ads.readADC_SingleEnded(2);

    // Umrechnen der Digits aus Spannung
    A0_Spannung = ads.computeVolts(A0_digits);
    A2_Spannung = ads.computeVolts(A2_digits);
    U_PeltierLeerlauf = A0_Spannung - A2_Spannung;

    digitalWrite(PWM_PIN,HIGH);     // aktivere den NMOS
```

```
            char str_R[10];        // für höhere Auflösung, mehr Nachkommastellen
            dtostrf(R, 4, 3, str_R); //double to string;
            //dtostrf(float_val,min_width,num_digits_after_decimal,where_store_string)

            //Kreieren des Ausgabestrings im Serial Monitor
            sprintf(string,"%s;%s;%s;%s;%s;%s;%s;", String(actualtime/1000.0).c_str(),
                    String(U_PeltierLeerlauf).c_str(), String(U_Peltier).c_str(),
                    String(I_shunt).c_str(), str_R, String(T1).c_str(),
                    String(T2).c_str());
            Serial.println(string);

            DisplayMeasurementData( U_Peltier , I_shunt , R , delta_T );
            // Aufruf der Funktion DisplayMeasurementData

            //wenn actualtime in Sekunden größer als die Messzeit, done=true
            if((actualtime/1000.0)>messzeit)
              done = true;

            i++; //zählt 1 zu i für jede Messrunde

            delay(250);

        } while(!done); // mach Messung so lang bis done=true

        digitalWrite(PWM_PIN,LOW  ); //deaktiviere den NMOS
        Serial.println("Messung Fertig!");
        lcd.clear();  //cleart Display
        lcd.setCursor(1,0);
        lcd.print("Messung fertig!");
        delay (2000);


}

//================================================================================
//Funktion DisplayMeasurementData
//================================================================================

//Eingabewerte float U_Peltier , float I_shunt , float R , float delta_T
void DisplayMeasurementData(float U_Peltier , float I_shunt , float R , float delta_T)
{
        lcd.clear();
        lcd.setCursor(0,0); //Position 1. Zeile, 1. Feld
        lcd.print("U_Peltier:");
```

```
        lcd.print(U_Peltier, 3); lcd.print("V ");

        //Ausgabe von I_Shunt
        lcd.setCursor(1,1); //Position 2. Zeile, 1. Feld
        lcd.print("I_Shunt:");
        lcd.print(I_shunt, 3); lcd.print("A ");

        //Ausgabe von R
        lcd.setCursor(4,2); //Position 3. Zeile, 1. Feld
        lcd.print("R:");
        lcd.print(R, 3); lcd.write((uint8_t)2);lcd.print(" ");
        // "lcd.write((uint8_t)2) fügt am Display Omega Symbol ein

        //Ausgabe von delta_T
        lcd.setCursor(3,3); //Position 4. Zeile, 1. Feld
        lcd.write((uint8_t)1);lcd.print("T:");
        // "lcd.write((uint8_t)1) fügt am Display Delta Symbol ein
        lcd.print(delta_T, 3); lcd.write((uint8_t)0);lcd.print("C");
        // "lcd.write((uint8_t)0) fügt am Display Grad Symbol ein
}


//================================================================================
//Starte void setup
//================================================================================

void setup() {

  lcd.begin(16,4);//Definition der Anzahl der Zeichen und Zeilen des LCD
  ads.begin(); //ADC ADS1115 Aufruf

  Serial.begin(57600); //Definition der Baudrate

  delay(5);

  pinMode(PWM_PIN,OUTPUT); //NMOS bereit für Empfang von Befehlen

  // definiere Grad-Symbol, Delta-Symbol, Omega-Symbol
  byte Celsius[8] = {B00111,B00101,B00111,B0000,B00000,B00000,B00000,B00000};
  //wenn 1, wird in Feld Pixel angesteuert (schwarz)
  lcd.createChar(0, Celsius);

  byte Delta[8] = {B00000,B00000,B00000,B00100,B01010,B10001,B11111,B00000};
  lcd.createChar(1, Delta);

  byte Omega[8] = {B01110,B10001,B10001,B10001,B10001,B01010,B11011,B00000};
  lcd.createChar(2, Omega);
```

```
  tempcheck(); //rufe Funktion tempcheck auf

  Serial.println("Ready!"); //wenn bei tempcheck alles passt -> gib ready aus

}

//=============================================================================================
//Funktion hasStringInside
//=============================================================================================

// From Jan's conversions Lib ( Kopiert von PID REgler Demo von Moodle )
// Überprüfe ob ein string im anderen string vorkommt

bool hasStringInside( const char *line , const char *str )
{
    if(line == NULL)
            return false;
    if( str == NULL)
            return false;

    int i = 0;
    int j = 0;
    int LineLenght = strlen( line );
    int strLen = strlen(str );
    int countcorrect = 0;

    // Loop through the "line" string
    for(  i = 0; i < LineLenght ; i++ )
    {
        countcorrect = 0; //Variable zurücksetzen, zum zählen
        if( line[i] == str[0] ) // if the first char of the searching word is correct
                {
                    for( j = 0; j < strLen ; j++ )
                      if( line[i+j] == str[j] )
                          countcorrect ++;

                    if( countcorrect == strLen )
                         return true;
                }
    } //vergleicht jedes einzelne Element aus line mit str, step by step
      //wenn alle Elemente gleich sind und die Länge stimmt -> true

return false;
}

//=============================================================================================
```

```
//Funktion getCharElementFromString
//================================================================================

// From Jan's conversions Lib
// holt string aus string -> alles ab dem Seperator(Leerzeichen)

char *getCharElementFromString( char *string , int element_count , char seperator )
{
        char *p = string;
        char *p_start = NULL;
        int counter = 0;
        int i = 0;

        if( string == NULL )
                return 0; //Defaultwert wenn nichts übergeben wird

        // Loop um Seperator zu finden
        //so lange element count größer als 0, suche nach dem Seperator
        while( (*(p)!= 0) && (element_count != 0) )
        {
                if( (*p)=='\n' || (*p)==0 )
                        return 0;
        // wenn Zeilenumbruch oder 0, dann gib 0 zurück
                if( (*p) == seperator )
                        element_count--;
                p++;
        } // wenn Seperator gefunden, element_count - 1 ; so lang bis element_count = 0

        // p ist Position nach Seperator
        p_start = p;

        // Suche "Länge" der Zahl aka Stellen
        while( (*(p)!= 0) && (*(p)!= seperator) && ((*p)!='\n') )
        {
                counter++;
                p++;
        }

        // allocate - has to be deleted by calling function!!!
        p = (char *)malloc( (counter+1) * sizeof( char ) );
        // check

        if( p == NULL )
                return 0; //wenn nichts nach Seperator, return 0
        // copy
        for( i = 0; i < counter ; i++ ) //Zahl in p abgespeichert
            p[i] = *p_start++;
```

```
        p[i] = 0;
        return p; //Zahl wird als character zurückgegeben
}

//================================================================================
//Funktion ProcessUART
//================================================================================

// Processing of the USB/UART Interface
// Important Note: Strings in the Arudino are Terminated with '\0' not with CR or LF
// while in serial interface the Termination of a line is performed with LF.

void ProcessUART() //Überprüfe ob neue Daten vorhanden und speichere die eingetippten
                   // Befehle in receivedChars ab  -> Suche mittels endMarker

{
    static byte ndx = 0; //static byte bleibt über Funktionsende hinaus bestehen
    // Übereinkunft zwischen Sender und Emfänger
    // ndx ist Anzahl der eingetippten Elemente
    char endMarker = '\n';     // Linefeed
    char rc;

    // Receiving from the Serial interfaces.
    while (Serial.available() > 0 && newData == false)
    { //Serial.available gibt die Anzahl der zum Lesen verfügbaren Bytes zurück
        // get Data character by character.
        rc = Serial.read(); //Eingabe im Serialmonitor (Messung 10 z.B)
        if (rc != endMarker)                    // endMarker choosen to be LF '\n'
        {
            receivedChars[ndx] = rc;  //Gehe Element für Element durch und suche \n
            ndx++;
            if (ndx >= numChars) {
                ndx = numChars - 1;
            }
        }
        else                 // wenn Endmarker gefunden -> string fertig
        {
            receivedChars[ndx] = '\0'; // Zurücksetzen des String
            ndx = 0;
            newData = true;
        }
    }
}

//================================================================================
//Funktion ProcessReceivedData
```

```
//=============================================================================================

// ProcessReceivedData - based on Hyperion 2v0 Project, Jan Enekel
// Parsing & Execution of UART commands

void ProcessReceivedData()
{
  bool understood = false;
  int i = 0;
  char *temp = strlwr(receivedChars);     // Alles in Kleinbuchstaben umwandeln
  char *wl_string = NULL;

  if (newData == true)          // Newdata wird true durch ProcessUART Funktion
  {
  // Identifikation des Arudino, hilfreich wenn mehrere Arudinos auf einem USB
          if(hasStringInside( temp , "whoareyou" ) && (understood == false))
          {
              Serial.println(MY_IDENTITY);
              understood = true;
          }

          //wenn messung und Messzeit in serial monitor eingegeben
          if(hasStringInside( temp , "messung" ) && (understood == false))
          {
              Serial.println(MY_IDENTITY);
              wl_string = getCharElementFromString( temp , 1 , ' ' );
              //holt sich vom String alles ab Leerzeichen -> Messzeit
              float messzeit = String(wl_string).toFloat(); //Messzeit in foat umwandeln
              free(wl_string); // Speicherfreigabe

              if( messzeit <= 0.0 ) //wenn Messzeit<0, Fehlermeldung
              {
                sprintf(string, "Zu kleine Messzeit, Messzeit muss > 0 sein!" );
                Serial.println(string);
              }
              else
              {
                sprintf(string, "Starte Messung mit %s[s] Messzeit!" ,
                        String(messzeit).c_str() );
                Serial.println(string);
                messen( messzeit ); // starte Funktion messen
              }

              understood = true; //Freigabe -> Ausbruch aus if-Schleife
          }

// Falls nicht messung oder whoareyou getippt wurde, Fehlermeldung
```

```
          if(understood == false)
            Serial.println("no valid command");

          newData = false;
  }
}

//============================================================================================
//Funktion DisplayZwischenMessung
//============================================================================================

// Während nicht gemessen wird, gibt DisplayZwischenMessung am Display delta T wieder
// ständige Displayausgabe

void DisplayZwischenMessung()

{
  float T1_nach = 0.0;
  float T2_nach = 0.0;

  T1_nach = tempsensor1.readTempC(); //Auslesen von Temperatursensor 1
  T2_nach = tempsensor2.readTempC(); //Auslesen von Temperatursensor 2

  float delta_T_nach = abs(T1_nach-T2_nach); //Berechne delta T nach


  //Überprüfe ob delta_T_nach größer als 1 grad Celsius
  //Überprüfe ob Temperatursensoren angesteckt sind
  if (delta_T_nach > 1.0)
  {
       lcd.clear();
       lcd.setCursor(2,0);
       lcd.write((uint8_t)1);lcd.print("T zu gross!");
       lcd.setCursor(0,1);
       lcd.print("Nicht Messbereit");

       lcd.setCursor(3,3); //Position 4. Zeile, 1. Feld
       lcd.write((uint8_t)1);lcd.print("T:");
       lcd.print(delta_T_nach, 3); lcd.write((uint8_t)0);lcd.print("C");
  }
  else if (isnan(T1_nach) && isnan(T2_nach)) //wenn beide abgesteckt
  {
       Serial.println("Sensor 1 und 2 Verbindung getrennt!");
       lcd.clear();
       lcd.setCursor(0,0);
       lcd.print("Beide Sensoren");
```

```
      lcd.setCursor(0,1);
      lcd.print("abgesteckt!");
  }
  else if (isnan(T1_nach)) //wenn T1 abgesteckt
  {
      Serial.println("Sensor1 Verbindung getrennt!");
      lcd.clear();
      lcd.setCursor(0,0);
      lcd.print("Sensor1");
      lcd.setCursor(0,1);
      lcd.print("abgesteckt!");
  }
  else if (isnan(T2_nach)) //wen T2 abgesteckt
  {
      Serial.println("Sensor2 Verbindung getrennt!");
      lcd.clear();
      lcd.setCursor(0,0);
      lcd.print("Sensor2");
      lcd.setCursor(0,1);
      lcd.print("abgesteckt!");
  }
  else
  {
      lcd.clear();
      lcd.setCursor(2,0);
      lcd.print("Messbereit!");

      lcd.setCursor(3,3); //Position 4. Zeile, 1. Feld
      lcd.write((uint8_t)1);lcd.print("T:");
      lcd.print(delta_T_nach, 3); lcd.write((uint8_t)0);lcd.print("C");
  }

  delay (500);

  }

//================================================================================
//Starte void loop
//================================================================================

void loop() {

  ProcessUART(); //Aufruf der Funktion ProcessUART
  ProcessReceivedData(); //Aufruf der Funktion ProcessReceiveData
  DisplayZwischenMessung(); //Aufruf der Funktion DisplayZwischenMessung

}
```

6.2 Python Code

```
# -*- coding: utf-8 -*-
"""
Created on Thu Jul  7 23:47:42 2022

Philipp Rohr, 11706159
Michael Oblak, 01619042
"""

# ============================================================================
# Laden der Libraries
# ============================================================================
from scipy.optimize import curve_fit
import numpy as np
import matplotlib.pyplot as plt
import serial
import time
plt.close()
# ============================================================================
# Function zum einlesen von Arduino
# ============================================================================
def write_read(x):
    arduino.write(x) #sendet dem Arduino eine Wert in Bytes

    time.sleep(0.8)
    text = arduino.readline() #liest Zeilen ein
    line = text.decode() #decodiert von Bytes in String
    word1 = line.split(';') # trennt die Elemente beim Semikolon
    return word1



# ============================================================================
# ======================START des Hauptprogramm===============================
# ============================================================================

arduino = serial.Serial(port='COM7', baudrate=57600, timeout=.2) #Bautrate muss
                                                #       mit Arduino übereinstimmen

time.sleep(1.5);

num = input("Bitte Messzeit eingeben in sekunden: ") #Input für die Messzeit in
                                                      #Sekunden
sendestring = "messung " + str(num) + "\n" #Sendestring für den Arduino;
                            #WICHTIG: Damit es funktioniert \n nicht vergessen
                            # damit die Kommunikation funktioniert
```

```
word1 = [] #Initialisieren der Variable
word = ''  #Initialisieren der Variable

while word != ['Messung Fertig!\r\n']: #läuft solange, bis diese Zeile kommt
                        #und bricht dann ab die Zeilen in Python einzulesen

    word = write_read(sendestring.encode()) #Aufruf der Funktion mit dem
                                            #Sendestring

    word1.append(word) #die eingelesenen Zeilen werden in word1 abgespeichert



Zeit   = []     #Initialisieren der Variablen
U_Leerlauf = []
U       = []
I       = []
R       = []
T1      = []
T2      = []


#Abspeichern in die einzelnen Variablen aus word1
for i in word1[3:-1]:
    Zeit.append(float(i[0]))
    U_Leerlauf.append(float(i[1]))
    U.append(float(i[2]))
    I.append(float(i[3]))
    R.append(float(i[4]))
    T1.append(float(i[5]))
    T2.append(float(i[6]))


delta_T = abs(np.array(T1)-np.array(T2)) #Berechnen von delta_T

# ============================================================================
# Start der Subplot
# ============================================================================
fig, axs = plt.subplots(3) #3 Subplots untereinander


# ============================================================================
# Exponential Fit
# ============================================================================
```

```
def func(x, a, b, c):
    return a * np.exp(-b * x) + c #fit Funktion Exponential

popt0, pcov0 = curve_fit(func, delta_T, R)# in popt sind die Koeffizienten a,b,c
popt1, pcov1 = curve_fit(func, delta_T, U_Leerlauf)



#axs[0] für subplot 0 und axs[1] für subplot 1 usw.

axs[0].plot(delta_T, func(delta_T, *popt0), 'k--', #plotten des Fits
          label='$fit: %+5.3f * e^{%-5.3f * x} %+5.3f$' % tuple(popt0))
axs[1].plot(delta_T, func(delta_T, *popt1), 'k--', #plotten des Fits
          label='$fit: %+5.3f * e^{%-5.3f * x} %+5.3f$' % tuple(popt1))

#============================================================================
#Polyfit
#============================================================================
order = 4 #Ordnung des Polynoms
fit_coeff = np.polyfit(Zeit,delta_T,order) #Koeffizienten des Polyfits
val = np.polyval(fit_coeff,Zeit) #y-Werte zum plotten des Fits
axs[2].plot(Zeit,val,'k--', label='$fit: %+5.1e * x^4  %+5.1e * x^3  %+5.1e * \
            x^2  %+5.1e * x  %+5.1e$' %tuple(fit_coeff)) #plotten
                                                                  #des Fits


#Ausgabe der Polynom-Koeffizienten in der Konsole
letters = ['a','b','c','d','e','f','g','h','i','j','k','l','m','n','o','p','q',
           'r','s','t','u','v','w','x','y','z']

print(' ')
print('Polynom',order,'Grades:')
print(' ')
print('|   Koeffizienten     |        Wert                ')
print('=================================================')

j = 0
for i in fit_coeff:
    print('|      '+letters[j]+':             |  ',i,'        ')

    j += 1


#============================================================================
# Plotten der Messdaten
#============================================================================
```

```
axs[0].plot(delta_T,R,'r*',label='Messpunkte')
axs[0].set(xlabel='$\Delta$T / °C')
axs[0].set(ylabel='$R$ / $\Omega$')
axs[0].grid('on')
axs[0].set_title('Widerstand des Peltierelements bei %s s' %num)
axs[0].legend()


axs[1].plot(delta_T,U_Leerlauf,'r*',label='Messpunkte')
axs[1].set(xlabel='$\Delta$T / °C')
axs[1].set(ylabel='$U_{Leerlauf}$ / V')
axs[1].grid('on')
axs[1].set_title('Leerlaufspannung des Peltierelements bei %s s' %num)
axs[1].legend()

axs[2].plot(Zeit,delta_T,'r*',label='Messpunkte')
axs[2].set(xlabel= '$t$ / s')
axs[2].set(ylabel='$\Delta T$ / °C')
axs[2].grid('on')
axs[2].set_title('$\Delta T$ bei %s s' %num)
axs[2].legend()

plt.subplots_adjust(hspace=0.4)

arduino.close() #Schließt den Serial Port zum Arduino
```

Literatur

[1] https://learn.adafruit.com/adafruit-mcp9808-precision-i2c-temperature-sensor-guide/pinouts, Aufgerufen am 25.07.2022.

[2] https://learn.adafruit.com/adafruit-4-channel-adc-breakouts/pinouts, Aufgerufen am 25.07.2022.

[3] https://www.xgadget.de/anleitung/arduino-lcd-ohne-potentiometer-anschliessen/, Aufgerufen am 25.07.2022.

[4] ENEKEL Jan, PID Regler Demo von Moodle und Hyperion 2v0 Project

[5] https://fritzing.org/, Aufgerufen am 25.07.2022.

7 Anhang

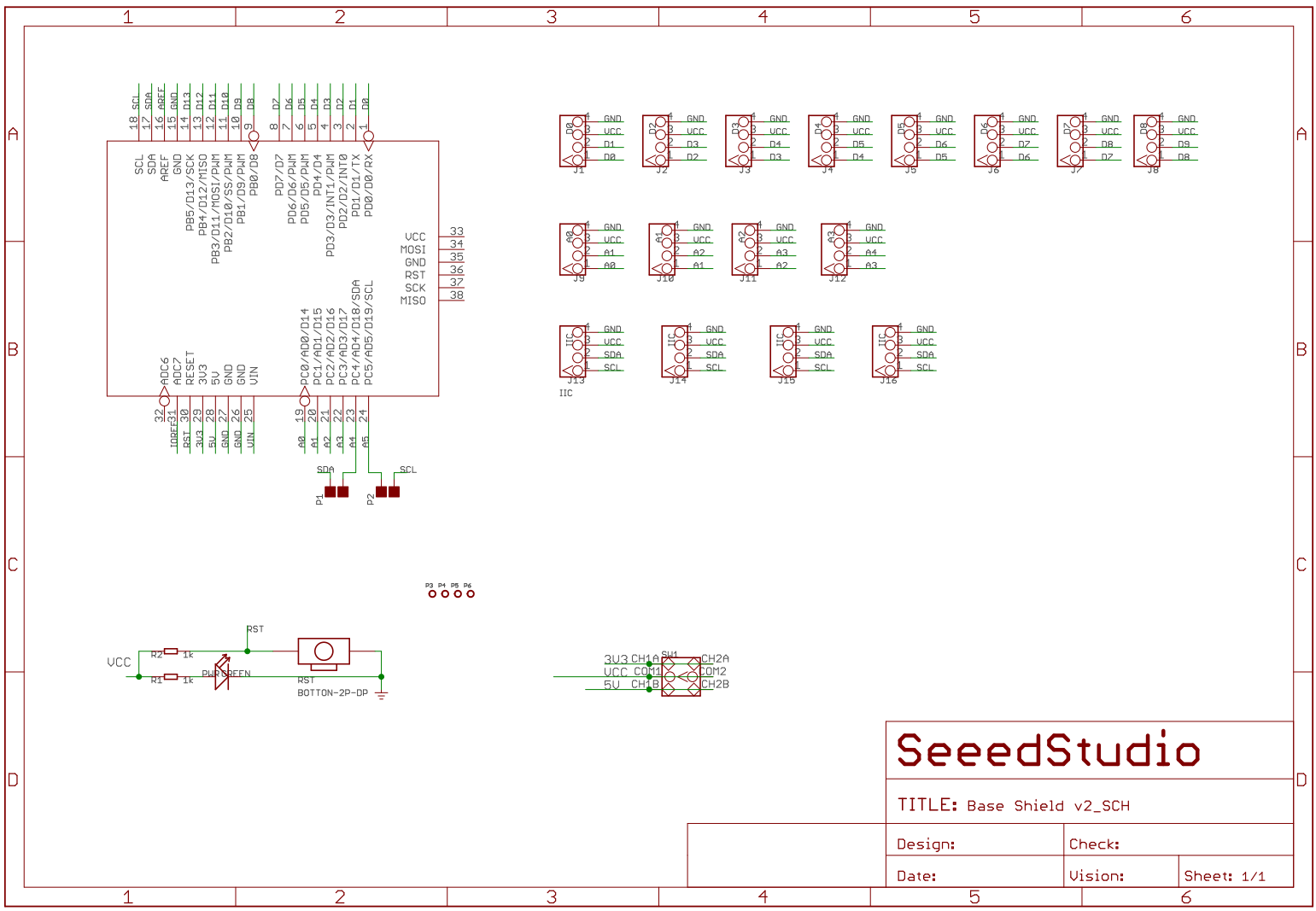
7.1 Datenblatt Base Shield

7.2 Datenblatt LCD-Module

7.3 Datenblatt NMosfet

7.4 Datenblatt Sub-D

7.5 Zeichnungen für 3D-Druck


SeeedStudio
TITLE: Base Shield v2_SCH
Design:
Check:
Date:
Vision:
Sheet: 1/1

MIDAS Displays
design • manufacture • supply

Electra House, 32 Southtown Road
Great Yarmouth, Norfolk
NR31 0DU, England

Telephone +44 (0)1493 602602
Fax +44 (0)1493 665111
Email:sales@midasdisplays.com
www.midasdisplays.com

FORDATA FOR DATA DISPLAY
Classic LCDs & LEDs

LCD MODULE SPECIFICATION

ITEM CODE
FC1604A01-RNNYBW-66SE

SPECIFICATION ESTABLISHED DATE: 2017.08.22

ISSUED BY: 朱叶 **CHECKED BY:** 李智良 **APPROVED BY:** 李智良

COPYRIGHT © 2000 ~ 2015 FORDATA ELECTRONIC CO., LTD. ALL RIGHTS RESERVED

INDEX

CONTENTS	PAGE
DATASHEET STATEMENT	1
CODE SYSTEM	2
GENERAL SPECIFICATIONS FEATURES MECHANICAL SPECIFICATIONS ABSOLUTE MAXIMUM RATINGS ELECTRONIC CHARACTERISTICS	3
LCD PANEL CHARACTERISTICS	4
LCD MODULE CHARACTERISTICS PIN ASSIGNMENT BLOCK DIAGRAM POWER SUPPLY DIAGRAM	5
FONT MAP	6
MECHANICAL DRAWING	7
PACKING DETAILS	8

AMENDMENT RECORD

MARK	DATE	DESCRIPTION	ITEM	PAGE	APPROVED
1	2017.08	INITIAL ISSUED	ALL	ALL	

1. The following icons are absolutely designed by FORDATA independently in 2007-SEP. They are unique in the LCD industry and are used for marking out FORDATA products' characteristics quickly and simply without any special meaning.
   FORDATA reserves the composing right and copyright. No one else is allowed to adopt these icons without FORDATA's approval.

2. The ISO9001 logo used in this document is authorized by SGS (www.sgs.com). FORDATA has already successfully passed the strict and professional ISO9001:2000 Quality Management System Certification and got the certificate. (No.: CN07/00404)

3. The technologies/techniques/crafts which denoted by the following icons are not exclusively owned by FORDATA but also shared by FORDATA's LCD strategic cooperators, however all these technologies/techniques/crafts have been finally confirmed by FORDATA's professional engineers and QC department.

4. As the difference in test standard and test conditions, also FORDATA's insufficient familiarity with the actual LCD using environment, all the referred information in this DATASHEET (including the icons) only have two functions:
   4.1: providing quick reference when you are judging whether the product meets your requirements or not.
   4.2: listing out the tolerance.
   FORDATA declares seriously: you should first test the corresponding sample(s) before signing the formal FORDATA SAMPLE APPROVAL document rather than consider this DATASHEET as the standard for judging whether the LCD meets your requirements or not . Once you place bulk order(s) to FORDATA without testing samples. FORDATA will disclaim all responsibility if the mass-production is proved not to meet with your requirements.

5. The sequence of the icons is random and doesn't indicate the importance grade.

6. Icons explanation

2000 Version 2006 Version 2012 Version

FORDATA FOR DATA DISPLAY

classic mono LCDs

Classic LCDs & LEDs

FORDATA is an integrated manufacturer of flat panel display (FPD). All above listed icons and words compses FORDATA's logo.
From 2000, FORDATA has supplied LCD module
From 2006, FORDATA has supplied TN, HTN, STN, FSTN monochrome LCD panel
From 2012, FORDATA has supplied all kinds of LED backlight.

FAST RESPONSE TIME
This icon on the cover indicates the product is with high response speed; Otherwise not.

HIGH CONTRAST
This icon on the cover indicates the product is with high contrast; Otherwise not.

WIDE VIEWING SCOPE
This icon on the cover indicates the product is with wide viewing scope; Otherwise not.

RoHS COMPLIANCE
This icon on the cover indicates the product meets ROHS requirements; Otherwise not.

3TIMEs 100% QC EXAMINATION
This icon on the cover indicates the product has passed FORDATA's thrice 100% QC. Otherwise not.

Vlcm = 3.0V
This icon on the cover indicates the product can work at 3.0V exactly; otherwise not.

PROTECTION CIRCUIT
This icon on the cover indicates the product is with protection circuit; Otherwise not.

LONG LIFE VERSION
This icon on the cover indicates the product is long life version (over 9K hours guaranteed); Otherwise not.

Anti UV VERSION
This icon on the cover indicates the product is against UV line. Otherwise not.

EASY OPERATION TEMPERATURE
This icon on the cover indicates the product can have good contrast on one driving voltage in indicated operation temperature range .

TWICE SELECTION OF LED MATERIALS
This icon on the cover indicates the LED has passed FORDATA's twice strict selection which promises the product's identical color and brightness; Otherwise not.

N SERIES TECHNOLOGY (2008 developed)
FORDATA adopts new structure, new craft, new technology and new materials inside both LCD module and LCD panel to improve the "RainBow"

1	2	3	4	5	6	—	7	8	9	10	11	12	—	13	14	15	16
F	C	08	01	A	23	—	F	H	Y	Y	B	W	—	5	2	L	E

No.	REMARKS	DESCRIPTION
1	COMPANY ABBRAVIATION	F = FORDATA
2	STANDARD MODULE TYPE	C = Character type standard LCD module (COB version) G = Graphic type standard LCD module (COB version)
3	Character (FC series)	08, 10, 12, 16, 20, 24, 40, = Character number Per line
	Graphic (FG series)	80, 100, 120, 122, 128, 160 ... ... = Row Dots Quantity
4	Character (FC series)	01, 02, 04, = Character Lines
	Graphic (FG series)	32, 64, 80, 128, 160 ... ... =Column Dots Quantity
5	Serial Number	A~Z which is decided by the sizes of viewing area
6	Identifying Code	00~99 which is decided by all the other aspects for the same viewing area
7	Polarizer type	R = Positive Reflective F = Positive Transflective M = Positive Transmissive N = Negative Transmissive B = Super Black technology New!
8	Backlight type	N = No Backlight L = Array Type LED Backlight S = Edge Type LED Backlight (Standard version) H = Edge Type LED Backlight (Long life span version) New! E = EL backlight without Invertor F = EL backlight with Invertor C = CCFL backlight without Invertor T = CCFL backlight with Invertor
9	Backlight color	N = No Backlight Y = Yellow-Green W = White R = Red A = Amber C = Blue-Green B = Blue G = Green Q = RedGreenBlue three color New!
10	LCD panel type	T = TN H = HTN Y = Yellow-Green STN G = Gray STN B = Blue STN F = FSTN
11	Viewing angle	B = Bottom 6:00 T = Top 12:00 R = Right 3:00 L = Left 9:00
12	Operation temperature range	S = 0℃ ~ 50℃ (Single Supply Voltage) D = 0℃ ~ 50℃ (Dual Supply Voltage) W = -20℃ ~ 70℃ (Single Supply Voltage) H = -20℃ ~ 70℃ (Dual Supply Voltage) T = -30℃ ~ 80℃ (Single Supply Voltage) E = -30℃ ~ 80℃ (Dual Supply Voltage)
13	Driving Voltage Code (This code was updated from 2015-JAN-1ST)	

	Vlcm = 3.0V	Vlcm = 3.3V	Vlcm = 3.6V	Vlcm = 5.0V
Vled = Indicated Voltage*	P	R	X	Q
Vled = 4.2V	M	G	D	K
Vled = 3.0V	9	A	3	4
Vled = 3.3V	T	B	K	F
Vled = 5.0V	8	C	2	5
NO/EL/CCFL	1	H	7	6

14	Backlight Connect Method	0 = PIN1 LED-, PIN2 LED+ 1 = PIN15(17/19) LED+, PIN16(18/20) LED- 2 = PIN15(17/19) LED-, PIN16(18/20) LED+ 3 = PIN15(17/19) LED+, PIN16(18/20) NC 4 = PIN15(17/19) NC, PIN16(18/20) LED+ 5 = PINA LED+, PINK LED- 6 = No / EL / CCFL Backlight
15	IC Manufacturer Code	A~Z or 01~99 which is decided by different IC manufacturers
*16	Font Set	A~Z or 01~99 which is decided by different font maps

* Please refer INDICATED VOLTAGE of LED in Page4 and Page5.

FEATURES

AVAILABLE OPTIONS	CHARACTERISTICS	CODE	No.
DISPLAY FORMAT	16 Characters by 4 Lines	FC1604A01	1~6
POLARIZER OPTIONS	Positive Reflective	R	7
BACKLIGHT TYPE OPTIONS	No Backlight	N	8
BACKLIGHT COLOR OPTIONS	No Backlight	N	9
LCD PANEL OPTIONS	Yellow-Green STN	Y	10
VIEWING ANGLE OPTIONS	6:00 (Bottom)	B	11
TEMPERATURE RANGE OPTIONS	-20°C ~ 70°C, Single Supply Voltage	W	12
SUGGESTED DRIVING VOLTAGE	Vlcm = 5.0V	6	13
SUGGESTED LED DRIVING MODE	No Backlight	6	14
CONTROLLER ▲1	ST7066U	S	15
FONT MAP CODE	E Version	E	16
DRIVING DUTY	1/16	—	—
DRIVING BIAS	1/5	—	—

▲1 Please ask for datasheet of the mentioned controller from FORDATA or FORDATA's authorized distributors. You can find the related information including AC & DC characteristics, Write & Read Timing diagram, Instruction table and descriptions, DDRAM & CGRAM, Rest Function and so on from the datasheet of controller.

▲1 You can ask for the example of software program (C language) from FORDATA or FORDATA's authorized distributors.

MECHANICAL SPECIFICATIONS

OVERALL SIZE	87.0W x 60.0H	mm	THICKNESS	max 14.5	mm
VIEWING AREA	61.8W x 25.2H	mm	HOLE-HOLE	82.0W x 55.0H	mm
CHARACTER SIZE	2.96W x 4.76H	mm	CHARACTER PITCH	0.59W x 0.59H	mm
DOT SIZE	0.56W x 0.56H	mm	DOT PITCH	0.04W x 0.04H	mm

ABSOLUTE MAXIMUM RATINGS

ITEM	SYMBOL	CONDITION	MIN	TYP	MAX	UNIT
POWER SUPPLY (LOGIC)	Vdd	25°C	-0.3	—	7.0	V
POWER SUPPLY (LCD)	V0	25°C	Vdd -13.5	—	Vdd +0.3	V
INPUT VOLTAGE	Vin	25°C	-0.3	—	Vdd +0.3	V
OPERATING TEMPERATURE	Vopr	—	-20	—	70	°C
STORAGE TEMPERATURE	Vstg	—	-30	—	80	°C

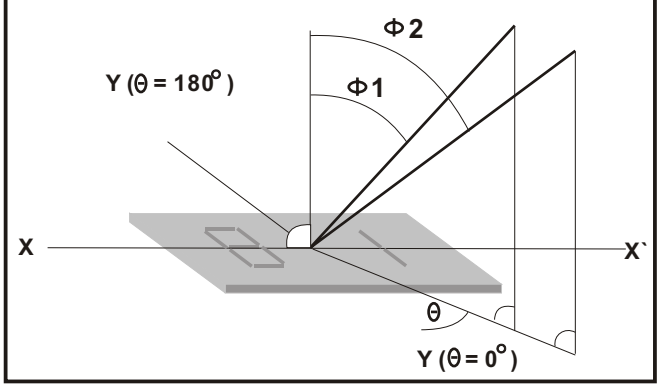
ELECTRONIC CHARACTERISTICS*

ICONS	ITEM	SYMBOL	CONDITION	MIN	TYP	MAX	UNIT
	INPUT VOLTAGE	Vlcm = Vdd	—	—	5.0	—	V
	SUPPLY CURRENT	Idd	Vdd=5V	—	1.5	—	mA
	DRIVING VOLTAGE FOR LCD PANEL	Vlcd = (Vdd - V0)	-20°C	4.40	—	4.70	V
			0°C	4.20	—	4.65	
			25°C	4.00	4.50	4.60	
			50°C	3.95	—	4.55	
			70°C	3.90	—	4.50	

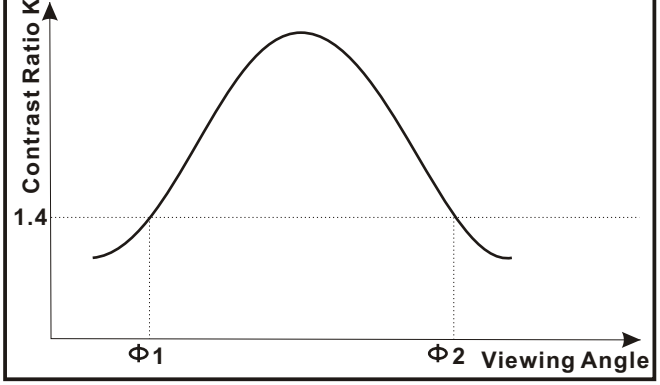
* All data are recorded from TEST REPORT #FSYP000700046

FOR STN/FSTN TYPE LCD Panel (TA=25°C, VIcd=5.0V ± 0.5V)							
ICONS	ITEM	SYMBOL	CONDITION	MIN	TYP	MAX	UNIT
	VIEWING ANGLE	Φ2 − Φ1	K=4	40	—	—	deg
		θ		60			
HC	CONTRAST RATIO	K	—	6	—	—	—
	RESPONSE TIME(RISE)	TR	—	—	150	250	ms
F	RESPONSE TIME(FALL)	TF	—	—	150	250	ms

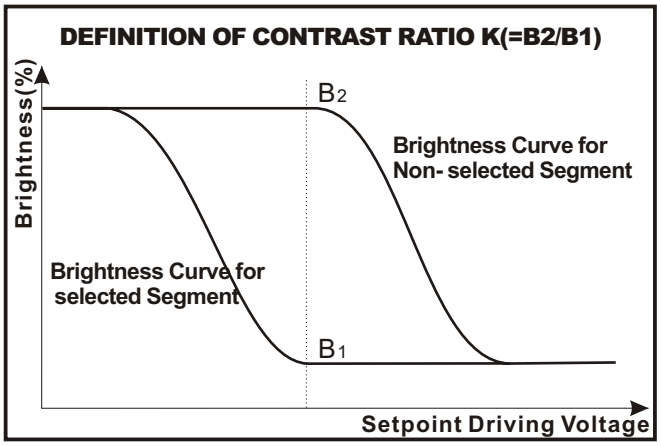
DEFINITION OF ANGLE Φ & θ



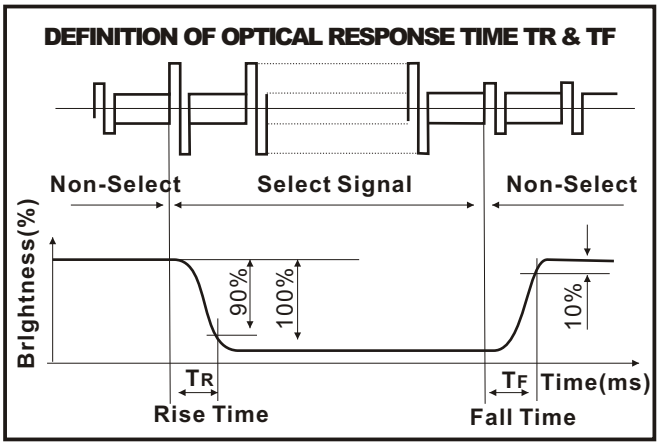
DEFINITION OF VIEWING ANGLE Φ1 &Φ2



DEFINITION OF CONTRAST RATIO K(=B2/B1)



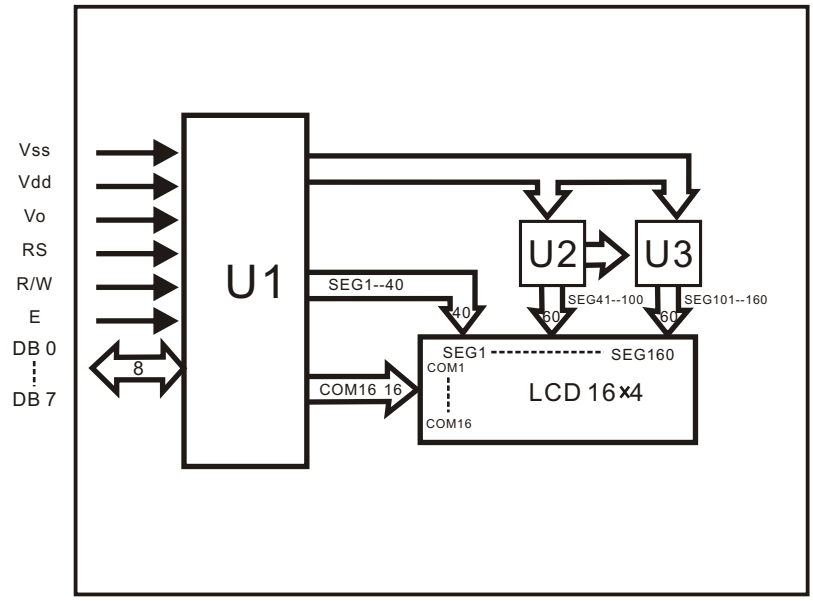
DEFINITION OF OPTICAL RESPONSE TIME TR & TF



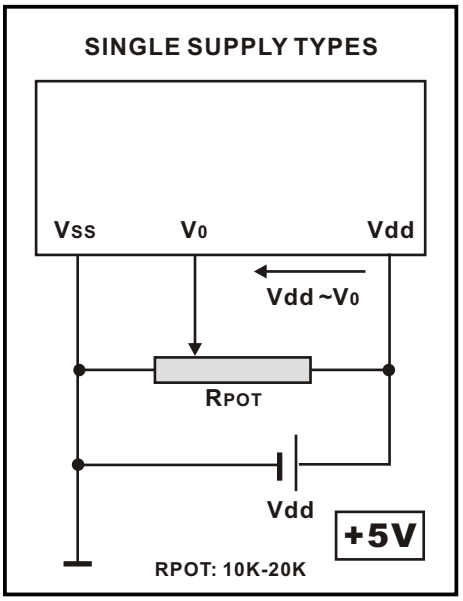
PIN ASSIGNMENT

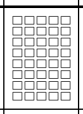
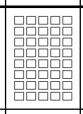
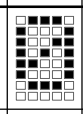
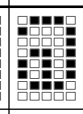
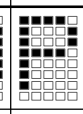
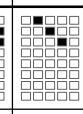
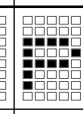
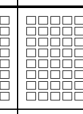
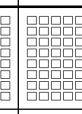
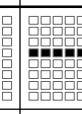
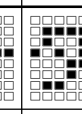
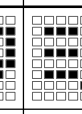
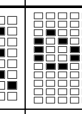
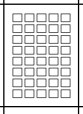
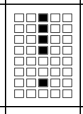
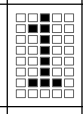
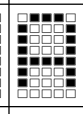
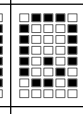
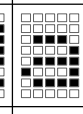
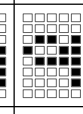
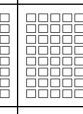
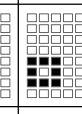
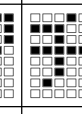
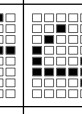
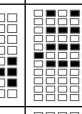
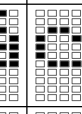
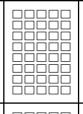
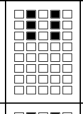
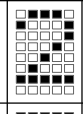
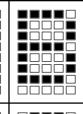
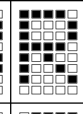
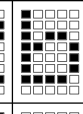
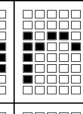
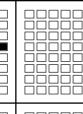
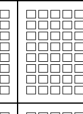
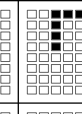
PIN	SYMBOL	DESCRIPTION	REMARKS
1	Vss	GND	
2	Vdd	Power supply for LCM	5.0V
3	V0	Contrast Adjust	
4	RS	Register Select Signal	
5	R/W	Data Read / Write	
6	E	Enable Signal	
7	DB0	Data bus line	
8	DB1	Data bus line	
9	DB2	Data bus line	
10	DB3	Data bus line	
11	DB4	Data bus line	
12	DB5	Data bus line	
13	DB6	Data bus line	
14	DB7	Data bus line	
15	NC	No connection	
16	NC	No connection	

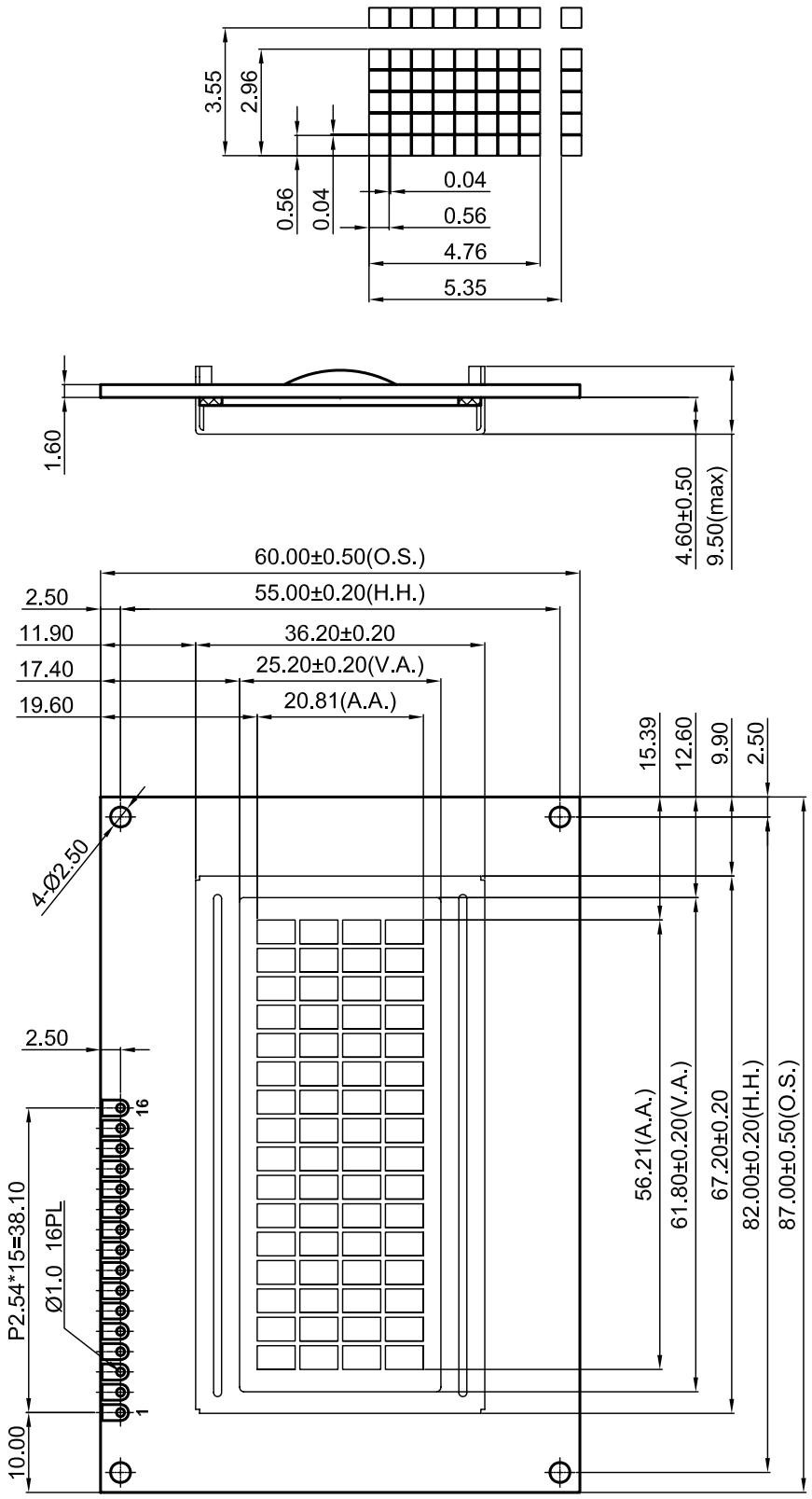
BLOCK DIAGRAM



POWER SUPPLY DIAGRAM



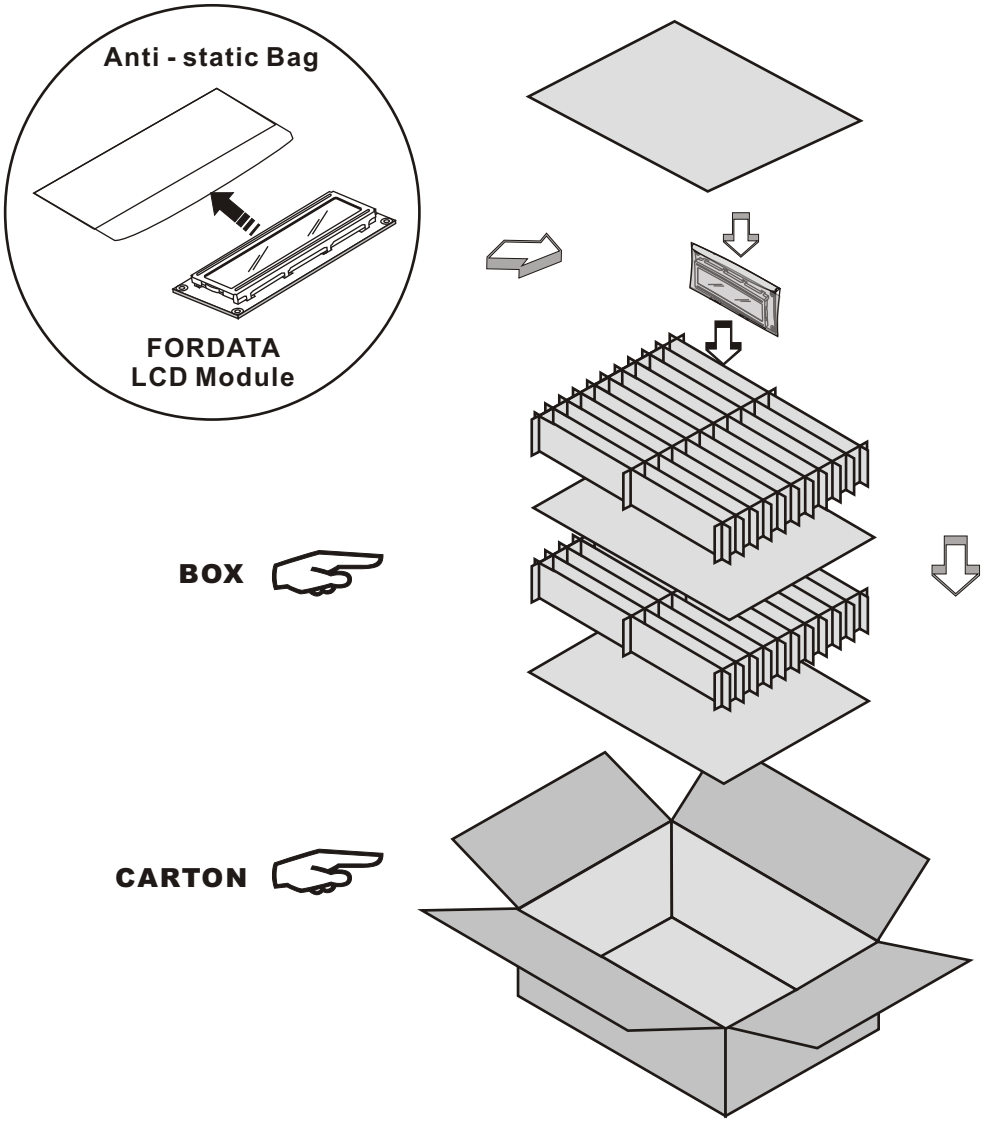
Upper 4bit / Lower 4bit	LLLL	LLLH	LLHL	LLHH	LHLL	LHLH	LHHL	LHHH	HLLL	HLLH	HLHL	HLHH	HHLL	HHLH	HHHL	HHHH
LLLL	CG RAM (1)															
LLLH	(2)															
LLHL	(3)															
LLHH	(4)															
LHLL	(5)															
LHLH	(6)															
LHHL	(7)															
LHHH	(8)															
HLLL	(1)															
HLLH	(2)															
HLHL	(3)															
HLHH	(4)															
HHLL	(5)															
HHLH	(6)															
HHHL	(7)															
HHHH	(8)															


3.55
2.96
0.56
0.04
0.04
0.56
4.76
5.35
1.60
4.60±0.50
9.50(max)
60.00±0.50(O.S.)
55.00±0.20(H.H.)
36.20±0.20
25.20±0.20(V.A.)
20.81(A.A.)
2.50
11.90
17.40
19.60
15.39
12.60
9.90
2.50
4-Ø2.50
2.50
16
1
P2.54*15=38.10
Ø1.0 16PL
10.00
56.21(A.A.)
61.80±0.20(V.A.)
67.20±0.20
82.00±0.20(H.H.)
87.00±0.50(O.S.)

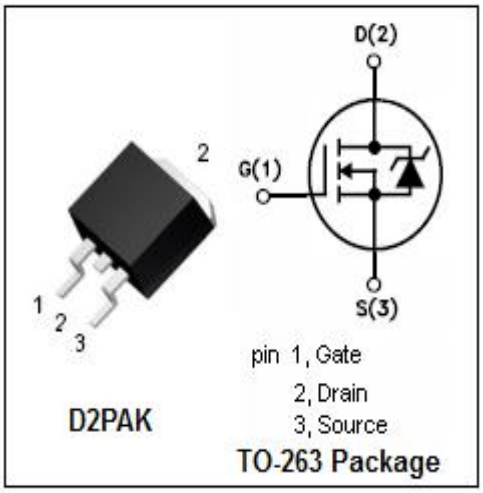
FULL-SIZED PACKAGE
30 PCS/BOX
8 BOXES/CARTON
240 PCS/CARTON
18.00 KGS/CTN(G.W.)
0.054 M^3/CARTON

HALF-SIZED PACKAGE
30 PCS/BOX
4 BOXES/CARTON
120 PCS/CARTON
9.00 KGS/CTN(G.W.)
0.027 M^3/CARTON

PACKING DECLARATION
1. This packaging information is for reference only. The actual information is subject to the actual packaging. Especially for packaging of LCL, tolerances may exist.
2. FORDATA will not be responsible for quality problems caused by unnormal transportation conditions (including but not limited to climate factors or human factors, such as improper handling).



Isc N-Channel MOSFET Transistor IRF540ZS



- **FEATURES**
- With To-263(D2PAK) package
- Low input capacitance and gate charge
- Low gate input resistance
- 100% avalanche tested
- Minimum Lot-to-Lot variations for robust device performance and reliable operation

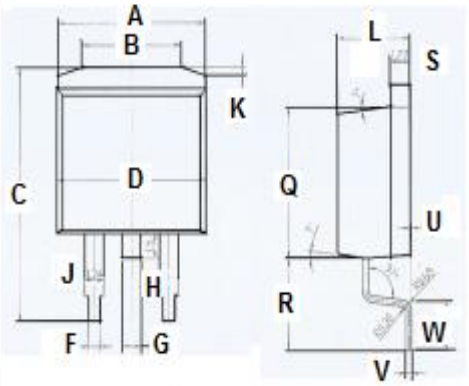
- **APPLICATIONS**
- Switching applications

- **ABSOLUTE MAXIMUM RATINGS(T_a=25℃)**

SYMBOL	PARAMETER	VALUE	UNIT
V_{DSS}	Drain-Source Voltage	100	V
V_{GSS}	Gate-Source Voltage	±20	V
I_D	Drain Current-Continuous Tc=25℃ Tc=100℃	36 25	A
I_{DM}	Drain Current-Single Pulsed	140	A
P_D	Total Dissipation @T_C=25℃	92	W
T_{ch}	Max. Operating Junction Temperature	175	℃
T_{stg}	Storage Temperature	-55~175	℃

- **THERMAL CHARACTERISTICS**

SYMBOL	PARAMETER	MAX	UNIT
Rth(ch-c)	Channel-to-case thermal resistance	1.64	℃/W
Rth(ch-a)	Channel-to-ambient thermal resistance	40	℃/W



DIM	mm MIN	MAX
A	10	
B	6.6	6.8
C	15.23	15.25
D	10.15	10.17
F	0.76	0.78
G	1.26	1.28
H	1.4	1.6
J	1.33	1.35
K	0.4	0.6
L	4.6	4.8
Q	8.69	8.71
R	5.28	5.30
S	1.26	1.28
U	0.0	0.2
V	0.37	0.39
W	2.80	2.82

Isc N-Channel MOSFET Transistor IRF540ZS

ELECTRICAL CHARACTERISTICS

T_C=25℃ unless otherwise specified

SYMBOL	PARAMETER	CONDITIONS	MIN	TYP	MAX	UNIT
BV_{DSS}	Drain-Source Breakdown Voltage	V_{GS}=0V; I_D= 0.25mA	100			V
$V_{GS(th)}$	Gate Threshold Voltage	V_{DS}=V_{GS}; I_D=0.25mA	2.0		4.0	V
$R_{DS(on)}$	Drain-Source On-Resistance	V_{GS}= 10V; I_D=22A		21	26.5	mΩ
I_{GSS}	Gate-Source Leakage Current	V_{GS}= ±20V;V_{DS}= 0V			±0.2	μA
I_{DSS}	Drain-Source Leakage Current	V_{DS}=100V; V_{GS}= 0V;Tj=25℃ V_{DS}=80V; V_{GS}= 0V;Tj=125℃			20 250	μA
V_{SDF}	Diode forward voltage	I_{SD}=22A, V_{GS} = 0 V			1.3	V

NOTICE：

ISC reserves the rights to make changes of the content herein the datasheet at any time without notification. The information contained herein is presented only as a guide for the applications of our products.

ISC products are intended for usage in general electronic equipment. The products are not designed for use in equipment which require specialized quality and/or reliability, or in equipment which could have applications in hazardous environments, aerospace industry, or medical field. Please contact us if you intend our products to be used in these special applications.

ISC makes no warranty or guarantee regarding the suitability of its products for any particular purpose, nor does ISC assume any liability arising from the application or use of any products, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages.

MERKMALE

- Robuste Konstruktion
- Durable Gehäuse aus verzinntem Stahl
- Vertiefungen am Gehäuse (nur Stecker) Für verbesserte elektrische Leitfähigkeit und Erdung
- Guter EMI/RFI-Schutz
- Vergoldete gestanzte und geformte Kontakte
- Flansche mit für Befestigungsbohrungen zur Schraubmontage an einem Platte
- Verzinnte Lötkelchanschlüsse für einfachen

15-poliger D-Sub-Steckverbinder für Frontplattenmontage von RS Pro, 2,77 mm Rastermaß

RS Best.-Nr. 544-3761



Professionelle Produkte von RS bieten Ihnen hochwertige Teile in allen Produktkategorien. Unsere Produktpalette wurde von Ingenieuren getestet und bietet eine vergleichbare Qualität wie die führenden Marken, ohne einen Premium-Preis zu zahlen.

Produktbeschreibung

Ein langlebiger D-Subminiatur-Steckverbinder mit einem robusten Gehäuse aus Stahl, gestanzte und gedrehte Crimpkontakte und Lötkelchen für den einfachen Anschluss von Drähten. Diese D-Sub-Steckverbinder sind ideal zur Herstellung von E/A-Verbindungen (Eingang/Ausgang) bei einer Vielzahl von Anwendungen in Industrie und Handel

Allgemeine Spezifikationen

Anzahl der Kontakte	15
Geschlecht	Stecker
Gehäuseausrichtung	Gerade
Montageart	Tafelmontage
D-Steckverbinder	Standard-D-Sub
D-Sub-Gehäusegröße	A
Anschlussmethode	Lot
Gehäusematerial	Stahl
Abmessungen	39,14 mm x 10,72 mm x 12,55 mm
Kontaktwerkstoff	Phosphorbronze
Kontaktbeschichtung	Gold
Anwendung	Computer, Monitore und Scanner, Netzwerk- und Kommunikationsgeräte; Datenverarbeitungsgeräte, Industrielle Geräte, Prüf- und Messgeräte, Fernbedienung Sensoren und Datenlogger, tragbare Geräte zur Stromerzeugung
Lebensdauer	150 Steckzyklen

Elektrische Spezifikationen

Nennstrom	5.0A
Nennspannung	5000V
Kontaktwiderstand	10 m Ohm MAX

Mechanische Spezifikationen

Rastermaß	2.77mm
Länge	39.14mm
Breite	10.72mm
Tiefe	12.55mm

Betriebsumgebungsspezifikationen

Maximale Betriebstemperatur	+105°C
Minimale Betriebstemperatur	-55°C

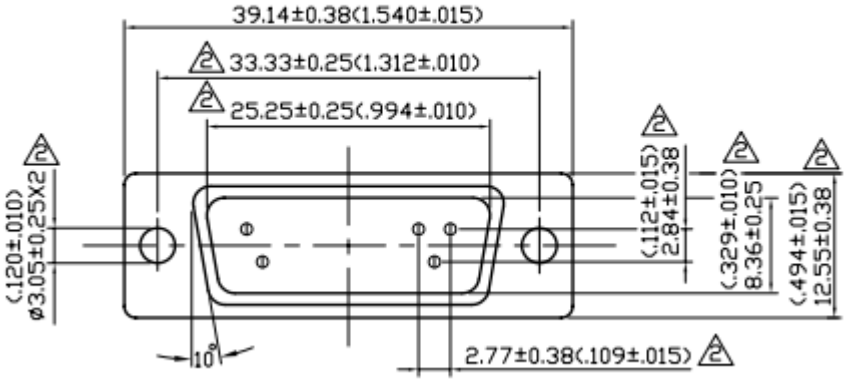
Zulassungen

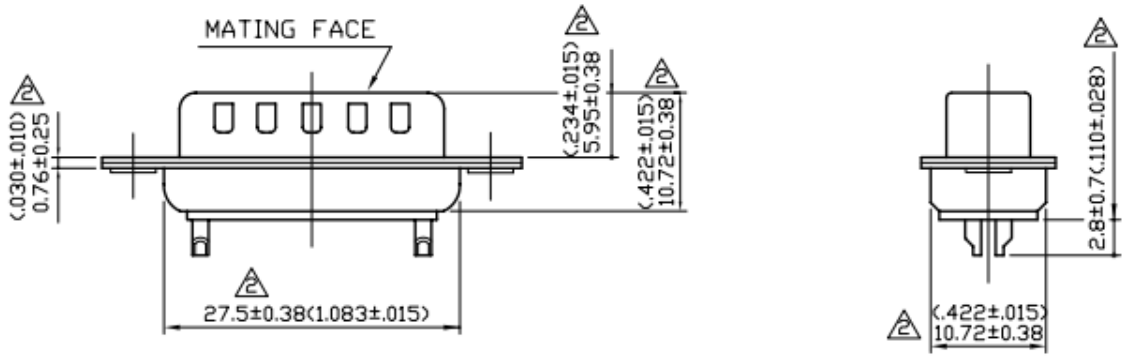
Konformität/Zertifizierung	2011/65/EU und 2015/863 / D








39.14±0.38(1.540±.015)
33.33±0.25(1.312±.010)
25.25±0.25(.994±.010)
(.120±.010)
ø3.05±0.25X2
(.112±.015)
2.84±0.38
(.329±.010)
8.36±0.25
(.494±.015)
12.55±0.38
10°
2.77±0.38(.109±.015)


MATING FACE
(.030±.010)
0.76±0.25
(.234±.015)
5.95±0.38
(.422±.015)
10.72±0.38
27.5±0.38(1.083±.015)
2.8±0.7(.110±.028)
(.422±.015)
10.72±0.38

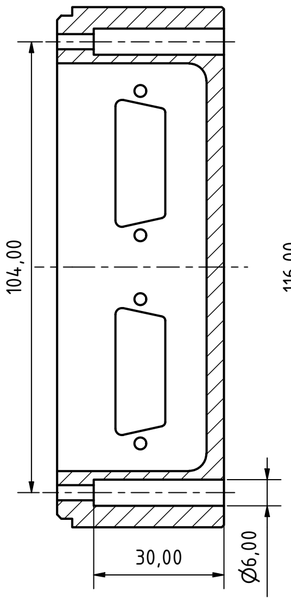
1

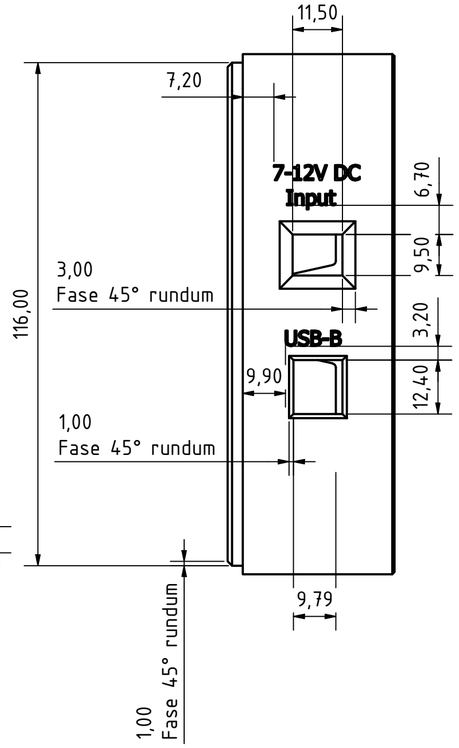
Schrift: Höhe: 3,5mm
Tiefe: 0.5mm
Schriftart: Tahoma

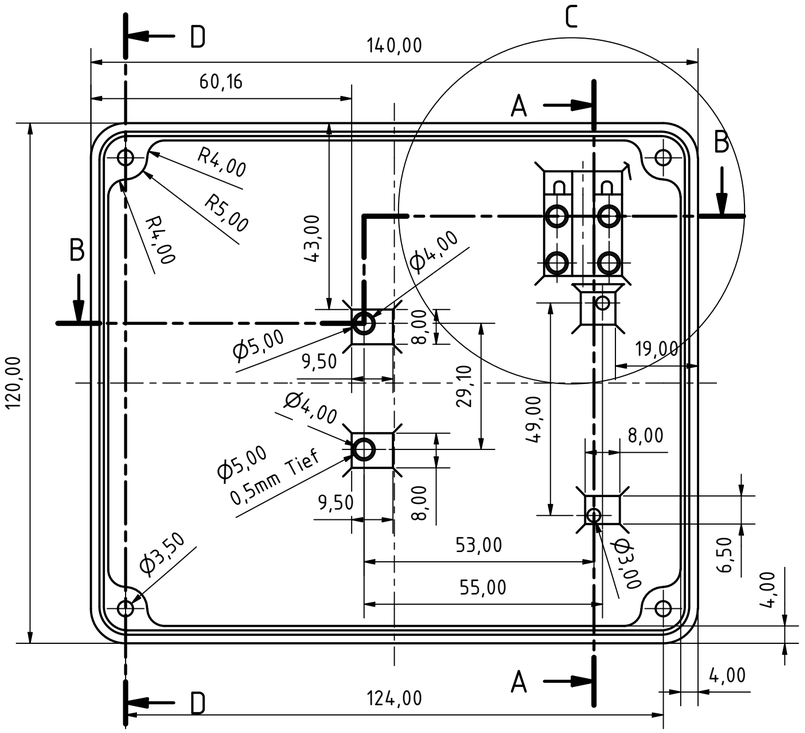
Alle Bohrungen: Bohrerauslauf zulässig

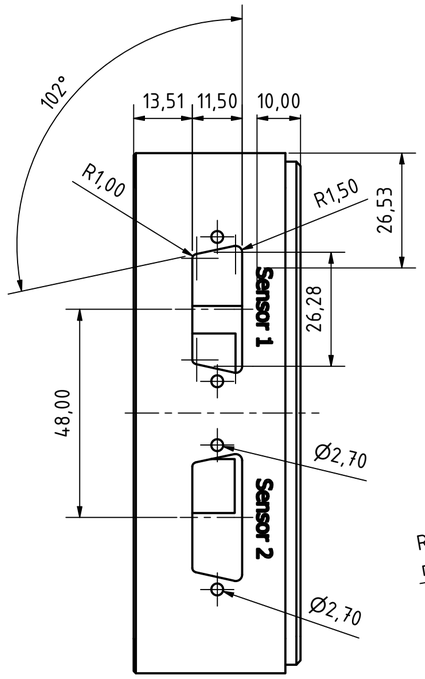
Alle nicht bemaßten Radien: R2mm

D-D (1 : 1)

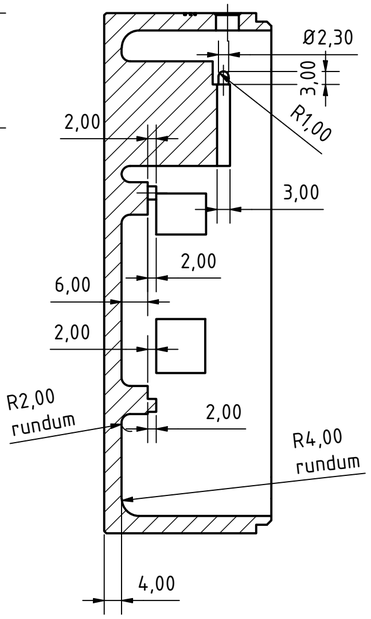


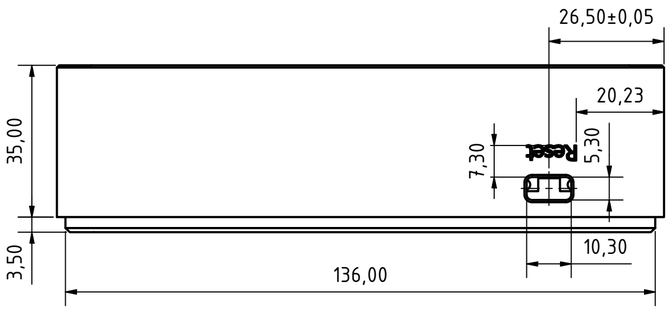




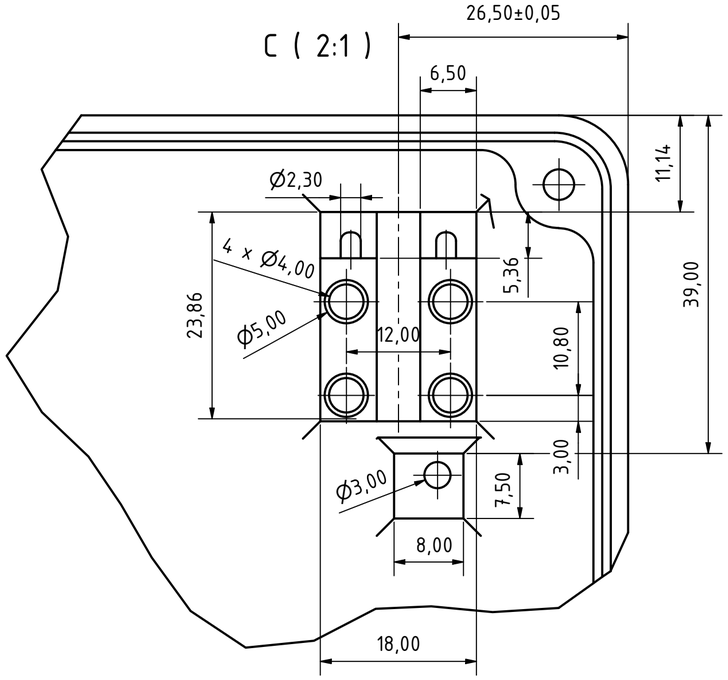


A-A (1 : 1)

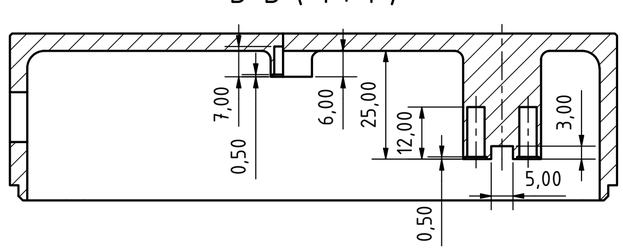




C (2:1)



B-B (1 : 1)



Ra1,6

TABELLE

Spalte 1	Spalte 2	Spalte 3	Spalte 4	Spalte 5
1	Gehäuseboden	1	PLA	3D-Druck
Position	Bezeichnung	Anzahl	Werkstoff	Hinweis

Allgemeintoleranz nach DIN ISO 2768-mk

0.8 0.3 0.5 0

Maßstab 1:1

	Datum	Name
Gezeich	17.07.2022	Oblak M.
Kontroll		
Norm		

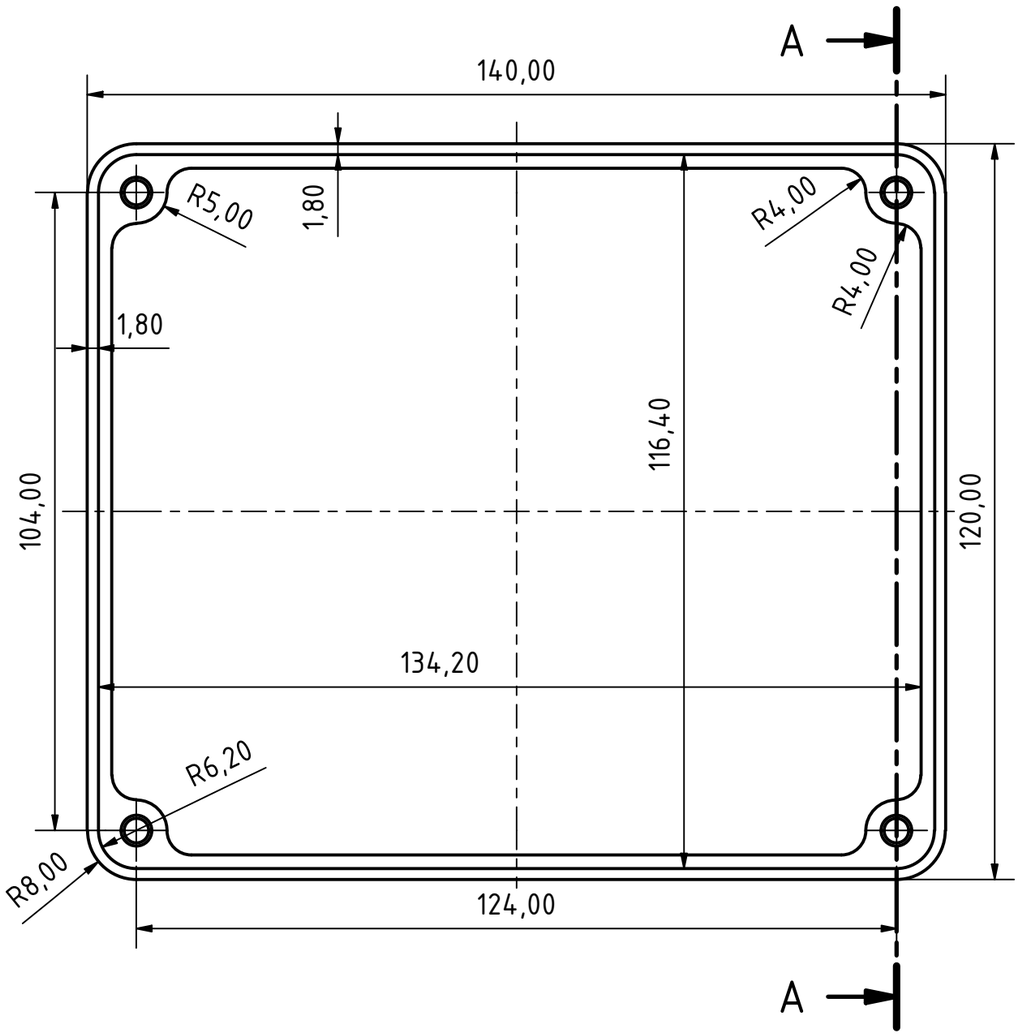
Gehäuseboden

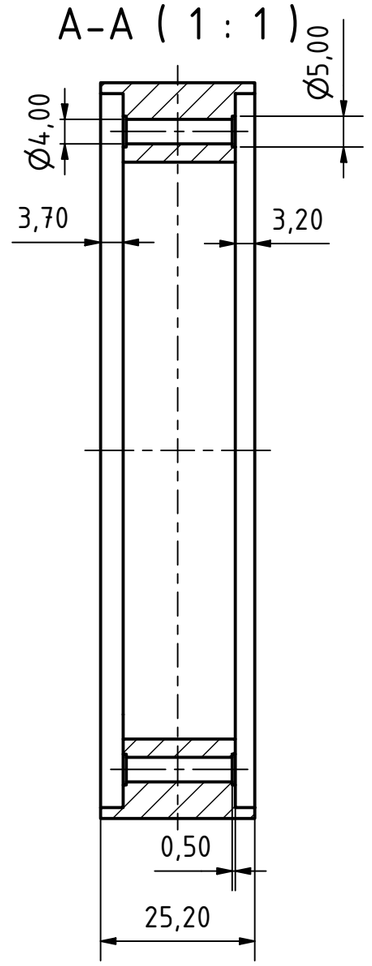
1

A2

Status	Änderungen	Datum	Name

2





Ra1,6

TABELLE

Spalte 1	Spalte 2	Spalte 3	Spalte 4	Spalte 5
2	Gehäusedeckelmittelteil	1	PLA-blau	3D-Druck
Position	Bezeichnung	Anzahl	Werkstoff	Hinweis

Allgemeintoleranz nach DIN ISO 2768-mk

0.8 0.3 0.5 0

Maßstab 1:1

	Datum	Name
Gezeich	22.07.2022	Oblak M.
Kontroll		
Norm		

Gehäusedeckelmittelteil

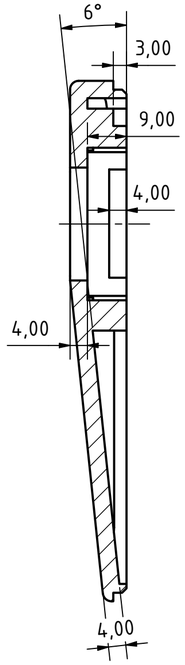
2

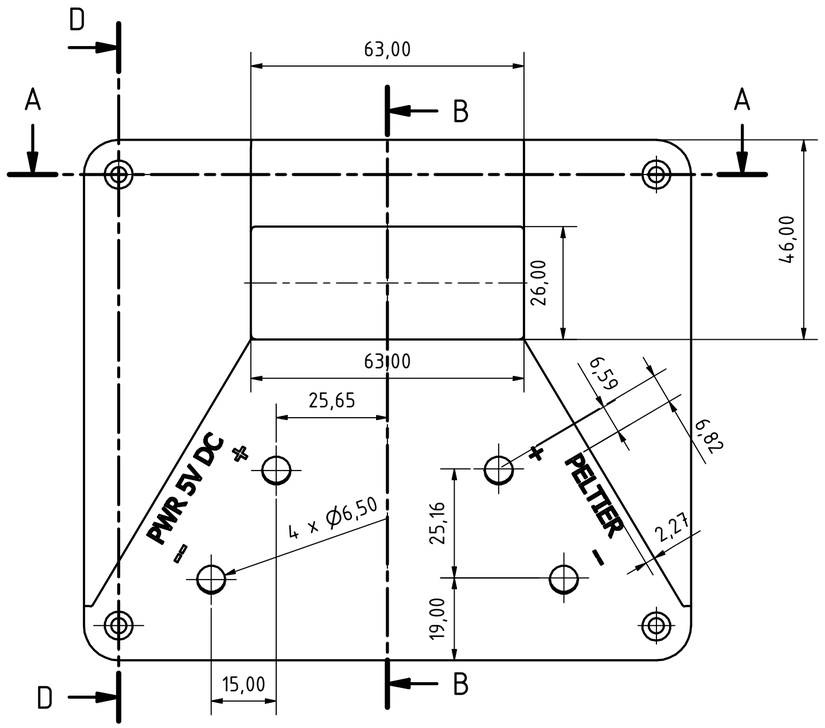
A3

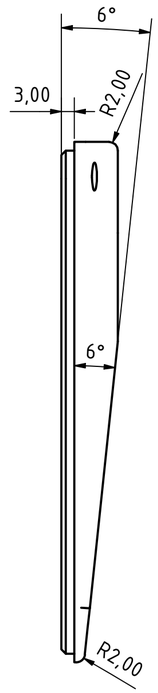
Status	Änderungen	Datum	Name

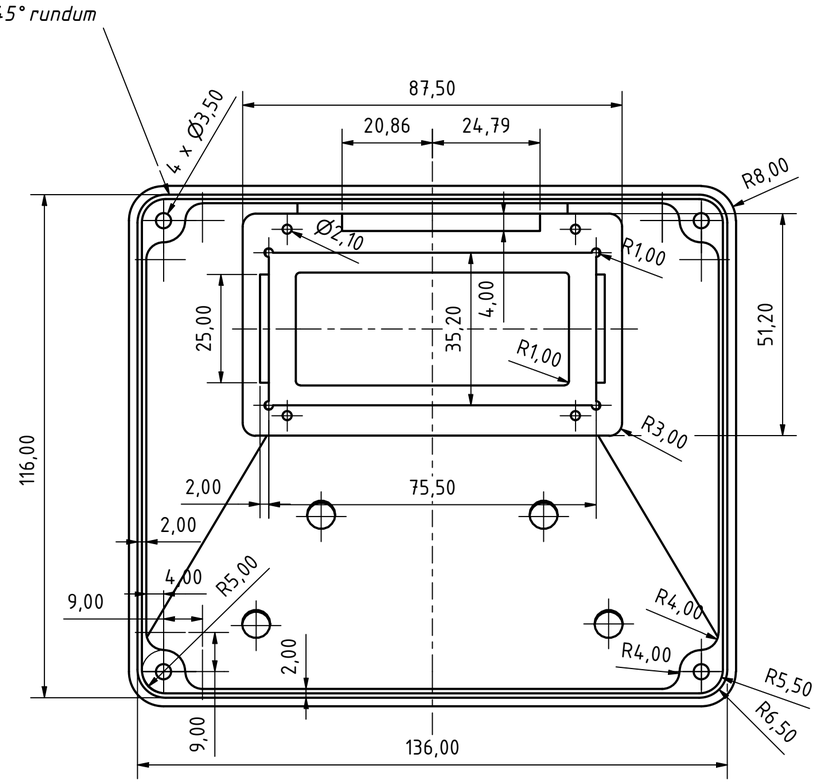
3

B-B (1 : 1)

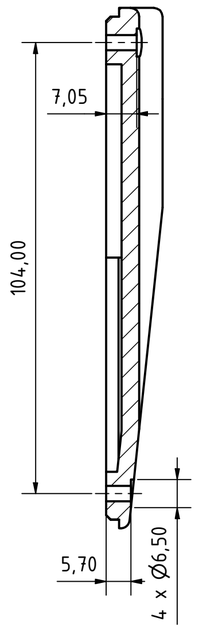


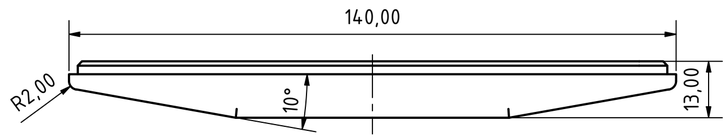




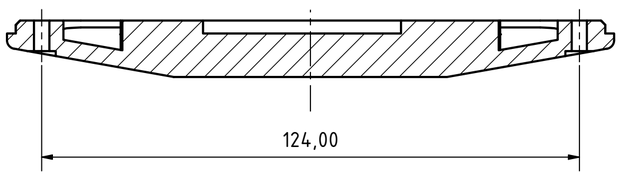


D-D (1 : 1)





A-A (1 : 1)



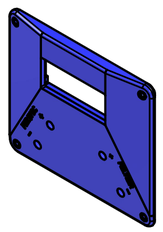


TABELLE				
Spalte 1	Spalte 2	Spalte 3	Spalte 4	Spalte 5
3	Gehäusedeckel	1	PLA-blau	3D-Druck
Position	Bezeichnung	Anzahl	Werkstoff	Hinweis

Allgemeintoleranz nach DIN ISO 2768-mk

0.8 0.3 0.5 0

Maßstab 1:1

	Datum	Name
Gezeich	22.07.2022	Oblak M.
Kontroll		
Norm		

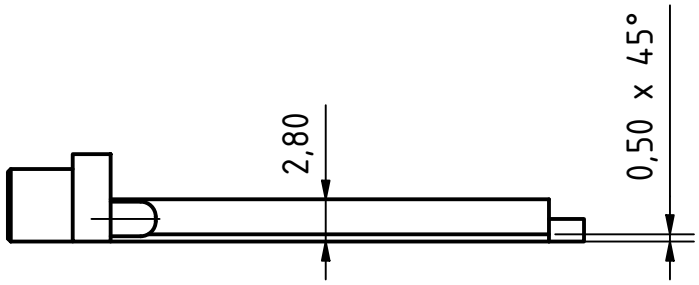
Gehäusedeckel

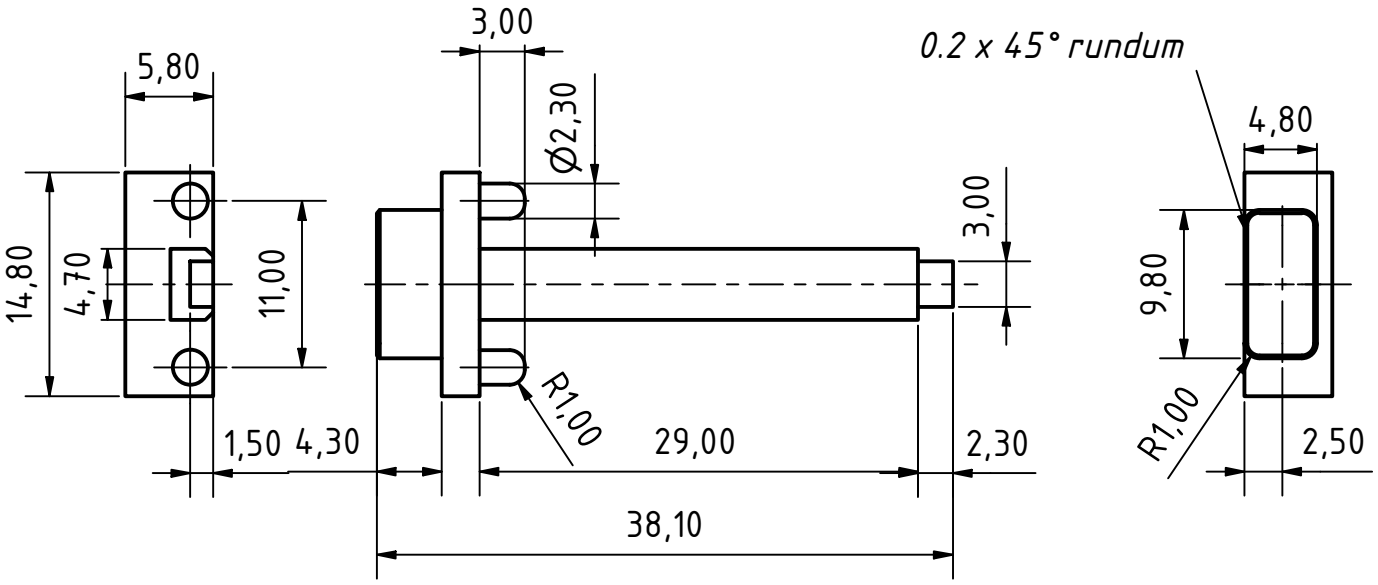
3

A2

Status	Änderungen	Datum	Name

4





TABELLE

Spalte 1	Spalte 2	Spalte 3	Spalte 4	Spalte 5
4	Schalter	1	PLA-rot	3D-Druck
Position	Bezeichnung	Anzahl	Werkstoff	Hinweis

Allgemeintoleranz nach DIN ISO 2768-mk

0.8 0.5
0.3 0

Maßstab 2:1

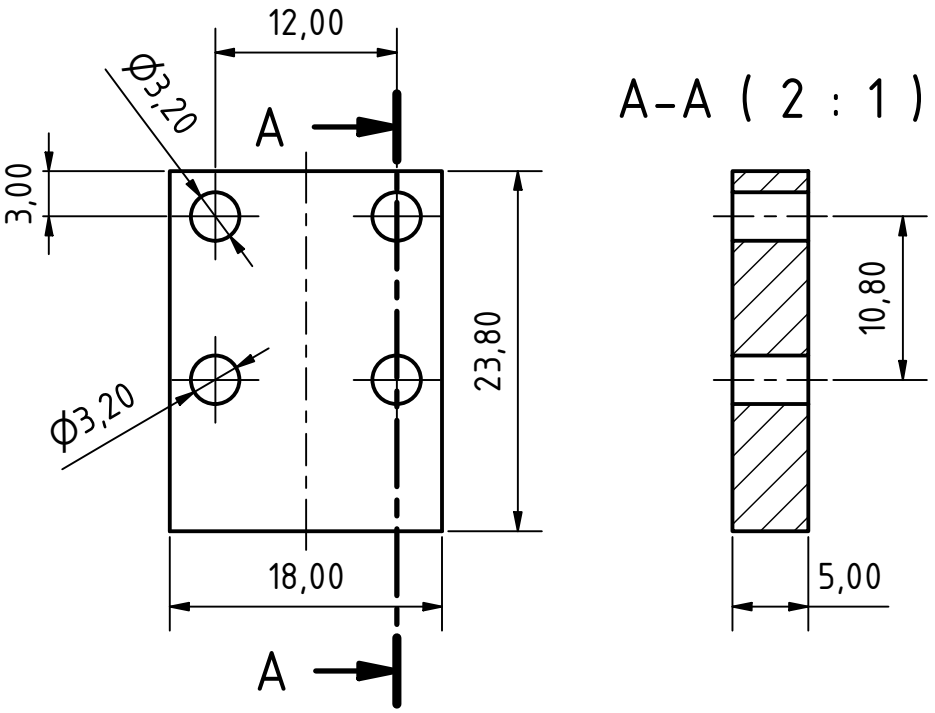
	Datum	Name
Gezeich	22.07.2022	Oblak M.
Kontroll		
Norm		

Schalter

1
A4

Status	Änderungen	Datum	Name

5



TABELLE

Spalte 1	Spalte 2	Spalte 3	Spalte 4	Spalte 5
5	Befestigungsblock	1	PLA-rot	3D-Druck
Position	Bezeichnung	Anzahl	Werkstoff	Hinweis

Allgemeintoleranz nach DIN ISO 2768-mk

0.8 0.3 0.5 0

Maßstab 2:1

	Datum	Name
Gezeich	23.07.2022	Oblak M.
Kontroll		
Norm		

Befestigungsblock

1

A4

Status	Änderungen	Datum	Name